



UNIVERSITY OF PISA

Department of Computer Science

ADVANCED PROGRAMMING

Academic Year 2025/2026

# A Minimal Deep Learning Framework Developed in Rust

Project Report

## Project Goal

Design and implementation of a minimal deep learning framework in Rust capable of training feed-forward neural networks on the **MNIST/California Housing** dataset, with a particular emphasis on advanced programming concepts covered during the course.

### Student

Alessio Pardini  
M.Sc. Computer Science  
Curriculum: Artificial Intelligence  
Student ID: 637814

### Course Professors

Andrea Corradini  
Antonio Cisternino

Academic Year 2025/2026

# Contents

|          |                                                                                                                              |           |
|----------|------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Problem Definition and Language-Agnostic Foundations of a Deep Learning Framework</b>                                     | <b>2</b>  |
| 1.1      | Operational Problem Definition . . . . .                                                                                     | 2         |
| 1.2      | From Necessity to Abstraction: A Constructive Derivation . . . . .                                                           | 3         |
| 1.2.1    | Consequence of (a): Structural Compositionality . . . . .                                                                    | 3         |
| 1.2.2    | Consequence of (b): Differentiability . . . . .                                                                              | 5         |
| 1.2.3    | Consequence of (c): Parameter Update . . . . .                                                                               | 10        |
| <b>2</b> | <b>Rust Implementation</b>                                                                                                   | <b>11</b> |
| 2.1      | Design of the <code>Module</code> Abstraction . . . . .                                                                      | 12        |
| 2.1.1    | Polymorphism as a Foundation for Interface Uniformity . . . . .                                                              | 12        |
| 2.1.2    | The <code>Module</code> Trait . . . . .                                                                                      | 12        |
| 2.1.3    | The Problem of Heterogeneous Composition . . . . .                                                                           | 13        |
| 2.1.4    | Parametric Polymorphism: Composition via Generics . . . . .                                                                  | 13        |
| 2.1.5    | Inclusion Polymorphism: Composition via Trait Objects . . . . .                                                              | 15        |
| 2.2      | Design of the <code>Tensor</code> Abstraction . . . . .                                                                      | 18        |
| 2.2.1    | Dimensional Compatibility Along the Chain . . . . .                                                                          | 18        |
| 2.2.2    | Representing a <code>Tensor</code> in Rust . . . . .                                                                         | 18        |
| 2.2.3    | The <code>data</code> Field . . . . .                                                                                        | 18        |
| 2.2.4    | The <code>shape</code> Field . . . . .                                                                                       | 19        |
| 2.2.5    | The <code>shape</code> Field . . . . .                                                                                       | 19        |
| 2.2.6    | The <code>strides</code> Field . . . . .                                                                                     | 19        |
| 2.2.7    | Ownership of <code>Tensor</code> Across <code>forward</code> , <code>backward</code> , and the Parameter Accessors . . . . . | 20        |
| 2.3      | Design of the <code>Loss</code> Abstraction . . . . .                                                                        | 24        |
| 2.4      | Design of the <code>Optimizer</code> Abstraction . . . . .                                                                   | 26        |
| <b>3</b> | <b>How I Used AI</b>                                                                                                         | <b>29</b> |
| 3.1      | Phase One: Designing the System . . . . .                                                                                    | 29        |
| 3.1.1    | The Trade-off: Speed versus Comprehension . . . . .                                                                          | 29        |
| 3.1.2    | A Valid Self-Critique . . . . .                                                                                              | 29        |
| 3.1.3    | Rapid Voice Prompting . . . . .                                                                                              | 29        |
| 3.1.4    | Cross-Checking Methodology: Validating the AI Output . . . . .                                                               | 30        |
| 3.2      | Phase Two: From Specification to Implementation . . . . .                                                                    | 32        |
| 3.2.1    | Setup: Claude Code as an Agentic Tool . . . . .                                                                              | 32        |
| 3.2.2    | Validation Strategy . . . . .                                                                                                | 32        |
| 3.2.3    | Personal Reflections on the Second Phase . . . . .                                                                           | 33        |
| <b>4</b> | <b>Appendix</b>                                                                                                              | <b>34</b> |
| 4.1      | Explicit derivation of the backward recursion . . . . .                                                                      | 34        |
| 4.2      | Algebraic Complexity and Dimensionality of VJP vs. JVP Accumulation . . . . .                                                | 36        |

# 1 Problem Definition and Language-Agnostic Foundations of a Deep Learning Framework

This chapter addresses a single question, deliberately independent of any programming language:

*what are the fundamental abstractions and mechanisms required to implement such a framework, distinguishing between the strict necessities dictated by the problem itself and the deliberate architectural choices that narrow the design space from that point on?*

Programming-language features, such as type systems, memory management, etc... are concerns of how an abstraction is realized, and are explicitly deferred to Chapter 2.

## 1.1 Operational Problem Definition

The operational formulation of the framework rests upon three fundamental pillars:

### Pillar 1: Parametric Representation and Structural Compositionality

A feed-forward neural network is formally defined as a parameterized mapping

$$f_\theta : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad \theta \in \mathbb{R}^p, \quad (1)$$

constructed as a sequence of  $L$  functional compositions:

$$f_\theta = g_{\theta_L} \circ g_{\theta_{L-1}} \circ \cdots \circ g_{\theta_1}, \quad (2)$$

where  $L \in \mathbb{N}_{\geq 1}$  denotes the structural depth (total number of layers) of the network architecture, and each  $g_{\theta_l}$  represents an elementary layer transformation comprising linear transformations and non-linear activation functions, parameterized by its own local parameter subset  $\theta_l \in \mathbb{R}^{p_l}$ .

The global parameter vector  $\theta$  is thus itself compositional, decomposing as

$$\theta = (\theta_1, \theta_2, \dots, \theta_L), \quad p = \sum_{l=1}^L p_l, \quad (3)$$

*Notation.* Throughout the text we refer to the same object,  $g_{\theta_l}$ , using whichever form best fits the context:  $g_l$  when referring to the layer as a position in the composition, and  $g(\cdot; \theta_l)$  when differentiating with respect to  $\theta_l$ , since this form makes explicit the argument being differentiated.

### Pillar 2: Objective Formulation via Empirical Risk Minimization

Given an empirical dataset of  $N$  observations,  $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N \subset \mathbb{R}^n \times \mathbb{R}^m$ , the primary objective of training is the iterative minimization of the global empirical risk (or loss function)  $\mathcal{L}(\theta)$  over  $\mathcal{D}$ :

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(x_i), y_i), \quad (4)$$

where  $\ell : \mathbb{R}^m \times \mathbb{R}^m \rightarrow \mathbb{R}_{\geq 0}$  is a scalar sample-wise loss function measuring the discrepancy between the network prediction  $f_\theta(x_i)$  and the ground-truth target  $y_i$ .

### Pillar 3: First-Order Gradient-Based Parameter Optimization

Optimization is typically driven by first-order gradient-based algorithms — SGD, momentum-based methods, and Adam all belong to this family, differing only in how the gradient  $\nabla_\theta \mathcal{L}_\mathcal{B}(\theta)$  is turned into a parameter update. We adopt **SGD** as the baseline algorithm:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_\mathcal{B}(\theta), \quad \mathcal{L}_\mathcal{B}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell(f_\theta(x_i), y_i), \quad (5)$$

where  $\eta > 0$  is the learning rate; taking  $\mathcal{B} = \mathcal{D}$  reduces Eq. (5) to full-batch Gradient Descent. This fixes the baseline algorithm, not the family of update rules the resulting abstraction must accommodate: I will later present SGD with Momentum and Adam as well, each of which requires per-parameter state persisting across steps ( $v_l$  for Momentum;  $m_l, v_l$  for Adam), whereas the stateless update of Eq. (5) does not — a distinction that directly motivates the design of that abstraction.

*Scope note.* Pillars 1–3 presuppose that a dataset  $\mathcal{D}$  is already available in memory, and that a batch  $\mathcal{B} \subseteq \mathcal{D}$  is supplied at each optimization step. How  $\mathcal{D}$  is loaded, and how  $\mathcal{B}$  is sampled or iterated over across steps, is an orthogonal concern, it does not bear on either Fact (a) or Fact (b), the two properties this chapter’s derivation is anchored to, and is therefore treated as external to the framework proper.

Two fundamental facts follow directly from this operational scope and govern the framework’s software architecture:

- (a) **Structural Compositionality:**  $f_\theta$  is defined as a composition of  $L$  elementary transformations, for arbitrary  $L \geq 1$  (Pillar 1).
- (b) **Differentiability:** minimizing  $\mathcal{L}(\theta)$  (Pillar 3) requires  $\nabla_\theta \mathcal{L}(\theta) \in \mathbb{R}^p$ . Since  $\mathcal{L}$  depends on  $\theta$  only through  $f_\theta$ , and  $f_\theta$  is compositional with  $\theta = (\theta_1, \dots, \theta_L)$  distributed layer-wise (Fact (a)), this gradient is computed *through* that composition, as a set of local gradients  $\{\nabla_{\theta_l} \mathcal{L}\}_{l=1}^L$  partitioning  $\nabla_\theta \mathcal{L}$ .
- (c) **Parameter Update:** minimizing  $\mathcal{L}_B(\theta)$  via Eq. (5) (Pillar 3) requires, at every step, revising  $\theta$  using  $\nabla_\theta \mathcal{L}_B(\theta)$  (Fact (b)). A mechanism must therefore exist that consumes this gradient and produces the next  $\theta$ .

Consequently, every design abstraction introduced hereafter must directly serve one, or both, of these fundamental properties.

## 1.2 From Necessity to Abstraction: A Constructive Derivation

### 1.2.1 Consequence of (a): Structural Compositionality

Fact (a) says that  $f_\theta$  is not an atomic function but a composition  $g_{\theta_L} \circ \dots \circ g_{\theta_1}$ . Stated that abstractly, it is easy to read past what it actually forces. Two observations make it concrete:

- The composite mapping  $g_{\theta_{l+1}} \circ g_{\theta_l}$  is itself a valid function, externally indistinguishable from a single primitive transformation  $g_{\theta_l}$ . This operational equivalence is essential for reasoning uniformly about networks of arbitrary depth  $L$ , regardless of how many internal transformations compose them.
- The output produced by  $g_{\theta_l}$  must be an object that  $g_{\theta_{l+1}}$  is able to consume as its input. If it cannot,  $g_{\theta_{l+1}} \circ g_{\theta_l}$  is simply undefined, and the composition Pillar 1 requires does not exist.

The first observation drives A1.

The second drives A2.

**A1 — A finitely specifiable mechanism for composing transformations.** Fact (a) states  $f_\theta = g_L \circ \dots \circ g_1$  for *arbitrary*  $L \in \mathbb{N}_{\geq 1}$  — Pillar 1 fixes no bound on depth. Any implementation, however, is by assumption specified by a finite set of rules. Checking this against increasing  $L$  makes the tension concrete:

- For  $L = 2$ , one can simply write  $g_2(g_1(x))$  by hand.
- For  $L = 3$ , one can just as easily write  $g_3(g_2(g_1(x)))$ .
- But no finite codebase can go on enumerating such expressions one  $L$  at a time, and certainly not for a depth  $L$  that is only known at runtime, configured by the user rather than hardcoded by the implementer.

Composition, therefore, cannot be defined by explicit enumeration over  $L$ : some finite mechanism must exist that constructs and evaluates a composition of whatever length it is given, without a separate case written out for each  $L$ . This is weaker than requiring a *single, uniform* mechanism, several finite mechanisms cooperating satisfy it just as well, provided none is itself indexed by  $L$ . **What Pillar 1 forces, together with finiteness, is only that such a mechanism (or cooperating set of them) exists, not that it be unique, nor any particular shape it must take.**

We call whatever satisfies this requirement a **composable transformation abstraction**, reserving the name **Module** for Chapter 2. (Trainability of the parameters inside such transformations is not assumed here, that comes once Fact (b) is derived.)

**A2 — A representation of data compatible across composition.** Consider the composition  $g_{l+1} \circ g_l$ : the value produced by  $g_l$  must be admissible as an input to  $g_{l+1}$ , or the composition is undefined. Pillar 1 fixes only the network’s two boundary spaces, input in  $\mathbb{R}^n$ , output in  $\mathbb{R}^m$ , and says nothing about the  $L - 1$  internal junctions:

- For  $L = 1$ , there is no internal junction:  $f_\theta = g_1$ , and Pillar 1’s boundary condition is sufficient.
- For  $L = 2$ , one junction appears: the output of  $g_1$  must be admissible as the input of  $g_2$ , a requirement Pillar 1 does not impose.
- For arbitrary  $L$ , there are  $L - 1$  such junctions, and the intermediate spaces need not share a dimension:

$$g_1 : \mathbb{R}^{784} \rightarrow \mathbb{R}^{128}, \quad g_2 : \mathbb{R}^{128} \rightarrow \mathbb{R}^{64}, \quad g_3 : \mathbb{R}^{64} \rightarrow \mathbb{R}^{10}.$$

What is forced, then, is not a shape shared by all transformations, but compatibility between each output and the next input, at every junction, including when a sub-composition is itself treated as a single transformation under A1’s operational equivalence. Pillar 1 supplies none of this; it follows entirely from requiring A1’s composition mechanism to be well-defined at each step. **This forces only the existence of a representation capable of carrying the exchanged numerical values while preserving that compatibility, not any particular data structure, memory layout, or evaluation strategy.**

I adopt the conventional term **Tensor** for this role: a representation of numerical values exchangeable across the network's transformations, whose shape varies with the domain and codomain of each. The name is conventional; the compatibility requirement is not.

### 1.2.2 Consequence of (b): Differentiability

Fact (b) requires computing the gradient of the batch empirical risk with respect to  $\theta$ . Throughout, we assume  $\ell$  and each  $g_l$  are differentiable in their trainable arguments, so that the composite map  $\theta \mapsto \ell(f_\theta(x_i), y_i)$  is differentiable and  $\nabla_\theta \mathcal{L}_\mathcal{B}(\theta)$  is well-defined:

$$\nabla_\theta \mathcal{L}_\mathcal{B}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_\theta \ell(f_\theta(x_i), y_i), \quad f_\theta = g_L \circ \dots \circ g_1, \quad (6)$$

where  $\mathcal{L}_\mathcal{B}(\theta)$  and  $f_\theta$  are exactly as fixed by Pillars 1 and 3.

Notice that while the sample-wise loss  $\ell : \mathbb{R}^m \times \mathbb{R}^m \rightarrow \mathbb{R}_{\geq 0}$  formally maps predictions and targets to a scalar, its dependence on the global parameters  $\theta \in \mathbb{R}^p$  is entirely channeled through the network's internal composition  $f_\theta(x)$ : for fixed  $(x_i, y_i)$ ,  $\ell(f_\theta(x_i), y_i)$  is itself a composite function of  $\theta$  alone. Evaluating  $\nabla_\theta \ell(f_\theta(x_i), y_i)$  therefore amounts to differentiating this composite mapping from the  $p$ -dimensional parameter space to a scalar.

By linearity of the gradient operator, it suffices to compute this gradient for a single, fixed input-output pair  $(x, y) \in \mathcal{B}$ : the sum over  $i \in \mathcal{B}$  then amounts to applying the same procedure once per example in the batch. *Fix one such pair  $(x, y)$  for the rest of this section.*

*To set the stage for our architectural requirements, I briefly formalize the standard backpropagation recursion.*

**The backward recursion.** Write  $z_0 := x$  and  $z_l := g_l(z_{l-1}; \theta_l)$  for  $l = 1, \dots, L$ , so that  $z_L = f_\theta(x)$  and  $\theta = (\theta_1, \dots, \theta_L)$ .

A direct application of the chain rule (CR) to this composition, worked out explicitly in Appendix 4.1, shows that  $\nabla_\theta \ell(z_L, y)$  is obtained by sweeping backward through the layers, carrying an *adjoint*  $a_l$ : a single vector encoding how sensitive the loss is to  $z_l$ .

**Seed** (nothing to propagate through yet):

$$a_L := \underbrace{\nabla_{z_L} \ell(z_L, y)}_{\text{sensitivity of the loss to its own input } z_L}. \quad (7)$$

**Step, for  $l = L, \dots, 1$ .**  $z_l$  depends on both  $\theta_l$  and  $z_{l-1}$ , so the incoming adjoint  $a_l$  is reused twice, once per dependency:

$$\begin{aligned} \text{local gradient (discarded after use):} & \quad \underbrace{\nabla_{\theta_l} \ell(z_L, y)}_{\text{sensitivity of the loss to } \theta_l} = \underbrace{\left( \frac{\partial g_l}{\partial \theta_l} \right)^\top}_{z_l \text{ vs. } \theta_l} \underbrace{a_l}_{\ell \text{ vs. } z_l} \\ \text{adjoint (carried to next iteration):} & \quad \underbrace{a_{l-1}}_{\ell \text{ vs. } z_{l-1}} = \underbrace{\left( \frac{\partial g_l}{\partial z_{l-1}} \right)^\top}_{z_l \text{ vs. } z_{l-1}} \underbrace{a_l}_{\ell \text{ vs. } z_l} \end{aligned}$$

One backward pass,  $L$  steps, every  $\nabla_{\theta_l} \ell$  recovered along the way.

We take this recursion and ask what it forces on an implementation.

**B1 — Which differentiation strategy?** As we have just seen formally, unrolling the recursion for  $\nabla_{\theta_l} \ell(z_L, y)$  yields a chain of matrix-Jacobian products, associative by construction:

$$\nabla_{\theta_l} \ell(z_L, y) = \left( \frac{\partial g_l}{\partial \theta_l} \right)^\top \left( \frac{\partial g_{l+1}}{\partial z_l} \right)^\top \dots \left( \frac{\partial g_L}{\partial z_{l-1}} \right)^\top \nabla_{z_L} \ell(z_L, y). \quad (7')$$

Associativity means this chain admits no privileged direction a priori: grouping the factors from the parameters outward (*forward accumulation*) and from the loss inward (*reverse accumulation*) are equally valid mathematically, and neither is forced by Eq. (7') alone.

What breaks the symmetry is the specific dimensionality regime fixed by Pillars 1–3: a massive parameter space ( $p \gg 1$ ) collapsing onto a *scalar* loss. Under this regime, Reverse accumulation relies on vector-Jacobian products (VJPs) that yield vectors directly, whereas forward mode requires matrix-matrix products at each step or, to avoid them, multiple sweeps per layer ( $\mathcal{O}(p)$  overall). We adopt reverse mode to avoid these prohibitive computational costs; a formal step-by-step comparison is provided in Appendix 4.2.

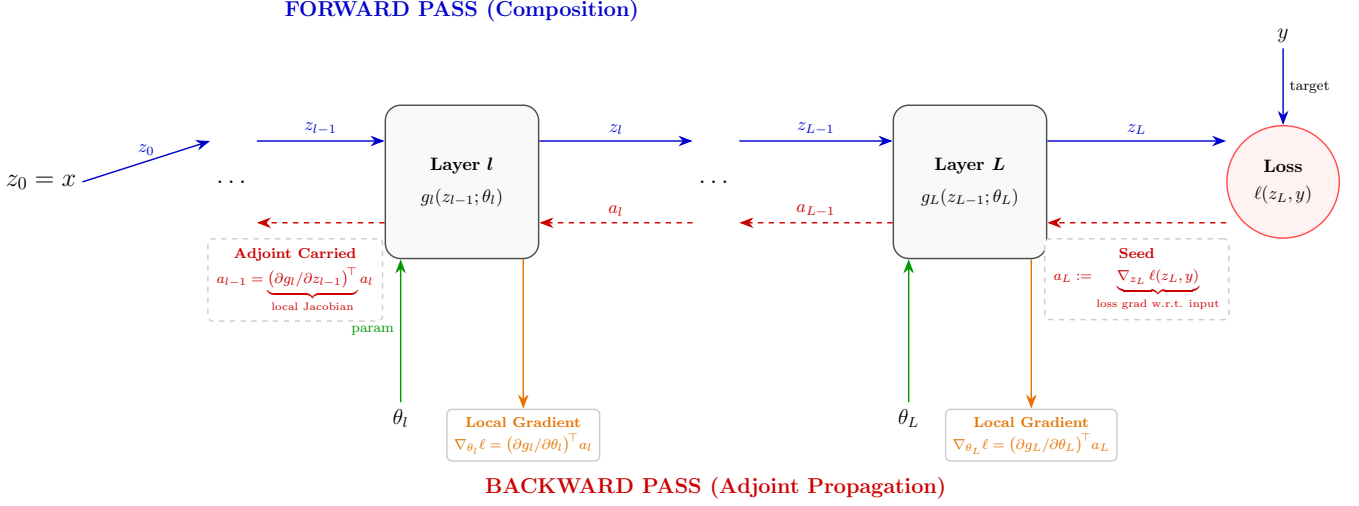


Figure 1: Computational graph illustrating the gradient computation via backpropagation. The *forward pass* (blue) computes and propagates the activations  $z$  from left to right to evaluate the loss. The *backward pass* (red) is seeded by the loss gradient and propagates the adjoint  $a$  from right to left. At each *step*  $l$ , the incoming adjoint  $a_l$  is reused twice: once multiplied by the local Jacobian w.r.t. the parameters to compute the local gradient (orange), and once multiplied by the local Jacobian w.r.t. the input to compute the next adjoint  $a_{l-1}$  to carry backward.

**From B1 to an Executable Recursion** B1 has settled *why* the recursion must be evaluated in reverse. It has not, however, settled *how* to *execute* it.

Carrying out the backward recursion at runtime requires two logically independent pieces of information, which I keep deliberately separate because they will be discharged by different mechanisms:

- (I) *knowing which steps to take* — i.e., the order in which the transformations  $g_1, \dots, g_L$  must be visited during the backward sweep;
- (II) *knowing how to carry out each step* — i.e., how to actually evaluate, at a given step  $l$ , the two vector-Jacobian products that produce  $\nabla_{\theta_l} \ell$  and  $a_{l-1}$  from  $a_l$ .

I address (I) as requirement B2 and (II) as requirement B3.

## B2 — Knowing Which Steps to Take: The Reverse Traversal Order.

*Statement.*

To execute the backward step at  $l$ , an implementation must have access to the ordering of the computations performed during the forward pass for the specific input  $x$  that produced  $z_L$ .

The substantive question that this statement raises is: *how* is this ordering made available at backward time? Answering it forces a choice between two positions well known in the automatic differentiation literature, usually opposed under the labels **define-and-run** and **define-by-run**.

Recall from A1 that *some* finite mechanism must exist for composing an arbitrary number  $L$  of transformations. That requirement was deliberately left open as to *how* such a mechanism represents the composition it constructs.

Two positions are compatible with it:

- If the finite mechanism of A1 is realized as an **explicit, immutable architecture**, fixed once, prior to any forward evaluation, and independent of the specific input  $x$ , then the backward ordering is obtained essentially for free: it suffices to traverse that same architecture in reverse. The information required by (I) is already there in full; *how* it is accessed (by index, by pointer, by explicit list) is an implementation detail deferred to Chapter 2. Optimizations aside, this is the essential idea.
- If, instead, A1 is realized so that the architecture’s own structure may vary with the input, the term “architecture” is admittedly imprecise here, but is retained for concreteness, then the ordering of computations is no longer recoverable from the architecture alone. A recurrent neural network is the canonical example: the effective depth of the unrolled computation depends on the length of the input sequence, so the network “architecture” must itself be unfolded anew for every input before its reverse traversal is even defined. Data-dependent branching is a second example of the same phenomenon.

*Selection.* I decided to adopt **define-and-run**.

Two reasons drive this choice:

- First, it is sufficient for the self-imposed scope of this project, whose target is training on 'MNIST/California Housing' with feed-forward architectures of fixed structure, the input-dependent unfolding that motivates define-by-run simply does not arise here.
- Second, and more importantly, the define-by-run alternative is intrinsically far more complex: it requires the composable-transformation mechanism of A1 to construct and record its own execution trace *while* it runs, rather than to be traversable by construction. Taking on that complexity would shift the project's center of gravity away from its stated purpose, an Advanced Programming exploration of what a language's abstraction mechanisms (ownership, type system, trait/typeclass dispatch) can express, and toward the engineering of a dynamic execution-tracing engine, which is a separate problem in its own right.

*Consequence for A1.* Fixing define-and-run changes what A1 must provide, not merely how it is used.

Until B2, A1 was required only to support *execution*: given an input, evaluate the composition forward. From this point on, A1 must additionally expose a **static, inspectable representation, traversable in reverse**.

Concretely:

*Additional requirement on A1.* A1 must provide an ordered, inspectable structure that, for every step  $l$ , guarantees the univocal association between the transformation  $g_l$ , its parameters  $\theta_l$ , and the preceding step  $l - 1$ . This association does not require that data be extracted outward from A1: it only requires that the structure itself always knows which transformation and which parameters are to be applied at that position of the reverse chain.

This closes requirement B2: the *order* of the backward sweep is now settled, and is discharged entirely by A1 in its define-and-run form.

### B3 — Knowing How to Perform Each Step: The VJP Operator.

Requirement B2 tells an implementation *which* layer to visit next; it says nothing about *how* to turn the incoming adjoint  $a_l$  into the outgoing pair  $(\nabla_{\theta_l} \ell, a_{l-1})$ .

That computation was already isolated in the backward recursion: running reverse accumulation at layer  $l$  amounts to evaluating two vector–Jacobian products (VJPs),

1. the parameter gradient,

$$\nabla_{\theta_l} \ell = \left( \frac{\partial g_l(z_{l-1}, \theta_l)}{\partial \theta_l} \right)^\top a_l,$$

2. the adjoint passed backward,

$$a_{l-1} = \left( \frac{\partial g_l(z_{l-1}, \theta_l)}{\partial z_{l-1}} \right)^\top a_l.$$

It is convenient to package both computations as a single operator, the **VJP of  $g_l$** , seen as a function of the quantities it needs and the pair it returns:

$$\text{VJP}_{g_l} : (\theta_l, z_{l-1}, a_l) \mapsto (a_{l-1}, \nabla_{\theta_l} \ell). \quad (8)$$

Equation (8) says nothing yet about *where*, in an implementation, this mapping is realized — only that some mechanism must realize it.

Two positions are available, and, as with B2, both are documented in the automatic-differentiation literature, where they are usually distinguished by *where the responsibility for computing a VJP resides*:

- **Internal VJP.** The mechanism of A1 that already represents  $g_l$  is itself responsible for computing  $\text{VJP}_{g_l}$ . Since A1 already owns  $g_l$  and  $\theta_l$  (by A1 and its B2 extension), no external communication is needed to obtain them.
- **External VJP.**  $\text{VJP}_{g_l}$  is computed by a mechanism separate from A1, which must then communicate with A1 to retrieve  $g_l$  and  $\theta_l$ , and again to write back any parameter update. This introduces communication overhead absent from the internal option. Its theoretical advantage is that a component with simultaneous visibility of *every*  $g_l$  and  $\theta_l$  can, in principle, perform cross-layer optimizations on the resulting derivative expressions, e.g. operator fusion, that a layer-local mechanism cannot see. Realizing such optimizations is itself a substantial and non-trivial engineering problem, and is not pursued here.

*Selection.* I adopt an **internal VJP**: the same mechanism that represents  $g_l$  under A1 is responsible for computing  $\text{VJP}_{g_l}$ . This is sufficient for the project's stated goal, and it avoids introducing, purely for the sake of a family of optimizations I do not intend to exploit, a second communication channel between A1 and an external differentiation component.

*Consequence for A1.* A1 must therefore be extended once more: in addition to executing  $g_l$  forward (base A1) and exposing an ordered, inspectable reverse structure (B2), it must also implement the backward operator  $\text{VJP}_{g_l}$  itself.



**Where Does the Parameter Update Belong?** Having just established that  $\text{VJP}_{g_l}$  is computed internally, by the same mechanism that already owns  $g_l$  and  $\theta_l$ .

This leaves one question open:

does that same mechanism also *apply* the update  $\theta_l \leftarrow \theta_l - \eta \nabla_{\theta_l} \mathcal{L}_{\mathcal{B}}$  (Pillar 3), or does it only *produce*  $\nabla_{\theta_l} \ell$  and stop there?

At first sight this looks like a trivial extension: if A1 already owns  $\theta_l$ , why not let it also overwrite  $\theta_l$  in place, with no further communication needed? Three concrete obstructions show this is not free.

- **Batching forces internal accumulation.** VJP is defined for a single pair  $(x, y)$ , producing  $\nabla_{\theta_l} \ell$  — but Pillar 3 requires the batch gradient  $\nabla_{\theta_l} \mathcal{L}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\theta_l} \ell_i$ . An update applied immediately after each single-example VJP is only correct if  $|\mathcal{B}| = 1$ ; for  $|\mathcal{B}| > 1$ , an internal update requires the mechanism to also accumulate  $\nabla_{\theta_l} \ell_i$  across  $|\mathcal{B}|$  backward sweeps and know *when* the batch is complete before applying anything.
- **The update rule is a free parameter.** A momentum-based or Adam-style rule computes the same  $\nabla_{\theta_l} \mathcal{L}_{\mathcal{B}}$  but combines it with additional per-parameter state,  $v_l$  (velocity) or  $(m_l, s_l)$  (first/second moment estimates), persisting across steps and shaped exactly like  $\theta_l$ . If the update is internal to A1, every instance of the composable-transformation mechanism must be told, individually, which rule and which hyperparameters ( $\eta$ , momentum coefficient,  $\beta_1, \beta_2, \dots$ ) to apply, and must carry whatever auxiliary state that rule needs.
- **Global operations on the gradient cannot be local to a single step.** Gradient clipping by global norm requires computing  $\|\nabla_{\theta} \mathcal{L}_{\mathcal{B}}\| = \left( \sum_{l=1}^L \|\nabla_{\theta_l} \mathcal{L}_{\mathcal{B}}\|^2 \right)^{1/2}$  *before* any single  $\theta_l$  is updated, since the scaling factor applied to each  $\nabla_{\theta_l}$  depends on gradients not yet produced at step  $l$  of a backward sweep still in progress. This conflicts with B1’s result: the  $\mathcal{O}(1)$  sweep produces all  $L$  local gradients within a single pass precisely because each step commits to its output immediately, with no second, read-only traversal to compute the global norm first. One could avoid this by clipping each  $\nabla_{\theta_l} \mathcal{L}_{\mathcal{B}}$  locally, as soon as it is produced, with a fixed per-layer threshold instead of the global norm, but this is a weaker heuristic, not an equivalent substitute: the right threshold is not known in advance, varies across layers, tasks, and datasets, and would itself become an additional hyperparameter to tune, trading one form of complexity for another rather than removing it.

*Selection.* I keep the update external to A1. A1 (via B3) is responsible for producing  $\nabla_{\theta_l} \ell$  and exposing it, alongside  $\theta_l$ ; a separate mechanism is responsible for accumulating gradients over  $\mathcal{B}$ , applying the update rule of Pillar 3, and owning whatever auxiliary state that rule requires.

*Consequence for A1.* A1 must be extended once more, minimally: it must expose  $\theta_l$  and the corresponding  $\nabla_{\theta_l} \ell$  to this external mechanism, a weaker requirement than applying the update itself, since no accumulation policy, update rule, or cross-layer reduction needs to be known by A1 to satisfy it.

satisfies this requirement an **Optimizer**, I will delve into the necessary abstractions in the section 1.2.3.

**The Missing Operand: Where Does  $z_{l-1}$  Come From?** Equation 8 is not yet fully specified: computing either VJP at step  $l$  requires the forward value  $z_{l-1}$ , and B2/B3 as stated so far say nothing about how it becomes available at backward time.

The underlying structural fact is unavoidable: the derivative of a non-linear function depends on the exact point at which it is evaluated, so neither Jacobian in Eq. 8 can be formed without plugging the concrete forward value  $z_{l-1}$  back into the derivative expressions. This produces a genuine temporal mismatch. During the backward sweep, the adjoint  $a_l$  arrives exactly when it is needed, from the right, it is, in a sense, a quantity from the future of the sweep. The operand  $z_{l-1}$ , by contrast, is an artifact of the forward pass, produced strictly in the past relative to the point where it is now required.

**How is this specific evaluation point retrieved?** Two alternatives are available.

- **Brute-Force Recomputation.** Since the original network input  $x$  is known throughout, the backward sweep can simply be paused at layer  $l$  and a partial forward pass re-run from  $x$  up to  $z_{l-1}$ .
  - *Pros:* negligible auxiliary memory,  $\mathcal{O}(1)$  — no intermediate state from the main forward pass need be retained.
  - *Cons:* computationally disastrous,  $\mathcal{O}(L^2)$  time overall: computing the gradient for an  $L$ -layer network requires re-evaluating increasingly long forward segments at every single backward step.
- **Smart Caching** Memory is accepted as cheaper than recomputation time. During the initial forward pass, as  $z_1, z_2, \dots$  are computed, they are not discarded but stashed in memory; when the backward sweep reaches layer  $l$ , it simply retrieves  $z_{l-1}$  from the cache.
  - *Pros:* computationally optimal,  $\mathcal{O}(L)$  time, VJP is evaluated as soon as its operands are retrieved.
  - *Cons:* memory heavy,  $\mathcal{O}(L)$  memory, training requires persistent memory proportional to network depth.

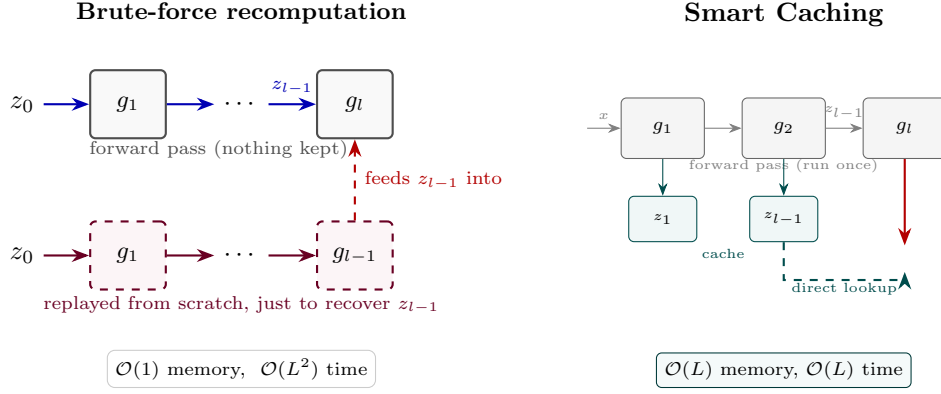


Figure 2: Recovering the missing operand  $z_{l-1}$  at backward step  $l$ . **Left:** nothing is retained from the forward pass, so  $z_{l-1}$  is recovered by replaying the forward pass up to layer  $l-1$  at every backward step ( $\mathcal{O}(1)$  memory,  $\mathcal{O}(L^2)$  time). **Right:** every  $z_l$  is cached once during the forward pass and retrieved by direct lookup ( $\mathcal{O}(L)$  memory,  $\mathcal{O}(L)$  time).

*Selection.* Between an  $\mathcal{O}(L^2)$ -time and an  $\mathcal{O}(L)$ -time strategy, and given that the  $\mathcal{O}(L)$  memory cost of Alternative B is entirely affordable for the network depths and MNIST-scale inputs, I decided to adopt **smart caching**: every  $z_l$  produced during the forward pass is retained until it is consumed by the corresponding backward step.

*Consequence for A1.* Since A1 is already the mechanism that executes  $g_l$  forward and produces  $z_l$ , retaining  $z_l$  is naturally its own responsibility: no external component has better standing to own this cache, and introducing one would only recreate, for  $z_{l-1}$ , the same unnecessary communication overhead already avoided for  $\theta_l$  in B3. A1 therefore also becomes responsible for caching, during the forward pass, exactly the values it will need to supply to its own backward operator.

The same reasoning disposes of the one remaining operand of Eq. (10): the adjoint  $a_l$  arriving *into* layer  $l$ 's backward step. Since the reverse traversal itself is owned by A1 (B2), and the VJP that consumes  $a_l$  is owned by A1 (B3), threading  $a_l$  from the output of layer  $l+1$ 's backward step into the input of layer  $l$ 's is again an entirely internal affair: no alternative in which this hand-off is managed from outside A1 is compatible with the ownership already established, so no further alternative needs to be enumerated here.

### Seeding the Backward Sweep: Where Does $a_L$ Come From?

So far, B2–B3 have settled how the backward sweep proceeds once it is running; they presuppose the existence of  $a_L = \nabla_{z_L} \ell(z_L, y)$  (Eq. 8) as already available. Nothing established so far produces it, a mechanism responsible to compute  $a_L$  from  $(z_L, y)$  must therefore exist, and be discharged by something.

*Two positions.*

- **Loss function as a terminal layer.** Consider  $\ell$  as  $g_{L+1}$ , such that  $a_L$  becomes the standard output of  $\text{VJP}_{g_{L+1}}$ , without requiring any new mechanism. This is not precluded by assumption A2 itself: A2 merely imposes compatibility of values at each junction point, not a specific calling interface. Thus, one could adapt that interface to meet this requirement, which, however, is necessary only for the loss layer, thereby cluttering it.
- **Loss as a separate abstraction.** Introduce a dedicated component, external to the  $g_1, \dots, g_L$  chain, whose responsibility is mapping  $(z_L, y)$  to whatever  $\ell(z_L, y)$  and  $a_L$  require.

*Selection.* I adopt the second position: a dedicated **Loss** abstraction, disjoint from the composable-transformation mechanism of A1. Given the interface fixed for  $g_1, \dots, g_L$ , this is the option that keeps that interface uniform and confines  $y$ -dependence to the one component that structurally requires it.

*Consequence.* The Loss abstraction is fixed by three requirements, each forced by what it must supply to B1–B3 without disturbing A1/A2:

- (L1) *Dual input.* To be well-defined, it must take both  $z_L$  and  $y$  as input.
- (L2) *Dual output.* The loss function must provide both the seed  $a_L$ —required by the backward pass—and the scalar  $\ell(z_L, y) \in \mathbb{R}_{\geq 0}$ .
- (L3) *Dimensional compatibility.*  $a_L$  is used by  $\text{VJP}_{g_L}$  (Eq. 8) as the input adjoint for the final step; this requires the shape of  $a_L$  to match exactly that of the other  $a_L - 1$ ,  $a_L - 2$ , etc.

Loss thus closes the last gap left open.

### 1.2.3 Consequence of (c): Parameter Update

Fact (c) requires that, at every step,  $\theta$  be revised using  $\nabla_{\theta}\mathcal{L}_{\mathcal{B}}(\theta)$  (Eq. 5). The preceding derivation already supplies every ingredient this revision needs: A1, extended by B3, exposes each pair  $(\theta_l, \nabla_{\theta_l}\ell)$  without applying any update itself; the subparagraph closing B3 gave three concrete reasons — batching, free choice of update rule, global operations on the gradient, why that application cannot be internal to A1.

What remains is to derive, from Fact (c) alone, exactly what the external mechanism performing that application must provide.

**C1 — Accumulation Across the Batch.** Eq. 5 requires  $\nabla_{\theta_l}\mathcal{L}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\theta_l}\ell_i$ , whereas a single backward sweep (B1–B4) produces only one per-example term  $\nabla_{\theta_l}\ell_i$ , for one fixed  $(x_i, y_i)$ . Obtaining  $\nabla_{\theta_l}\mathcal{L}_{\mathcal{B}}$  therefore requires  $|\mathcal{B}|$  independent sweeps, one per example, whose per-layer outputs  $\nabla_{\theta_l}\ell_i$  must be summed into a running total *as they are produced* and only then combined into the update.

This forces the existence of an accumulator, indexed like  $\theta$  itself by  $l = 1, \dots, L$ , that

- (i) starts at zero at the beginning of each batch,
- (ii) is incremented by  $\nabla_{\theta_l}\ell_i$  after every sweep  $i \in \mathcal{B}$ , and
- (iii) is read, once  $\mathcal{B}$  is exhausted, to form  $\nabla_{\theta_l}\mathcal{L}_{\mathcal{B}}$ .

**C2 — Access the Layers.** It must be possible to access the pairs  $(\theta_l, \nabla_{\theta_l}\mathcal{L}_{\mathcal{B}})$  for  $l = 1, \dots, L$ . No new design is required to achieve this access on the A1 side, as the B2 extension has already made it possible to expose an ordered, inspectable structure corresponding to  $l = 1, \dots, L$  for the backward pass.

**C3 — A Parametrized Update Rule with Local, Persistent State.** Pillar 3 takes SGD (Eq. 5) as the baseline, but leaves open other update rules, such as Momentum and Adam. These rules use the same batch gradient  $\nabla_{\theta_l}\mathcal{L}_{\mathcal{B}}$ , possibly together with auxiliary state that persists across update steps:  $v_l$  for Momentum, and  $m_l, v_l$  for Adam. For these standard optimizers, this state is maintained separately for each parameter block  $\theta_l$  and has the corresponding shape.

The mechanism discharging Fact (c) must therefore be parametrized by an update rule

$$u : (\theta_l, \nabla_{\theta_l}\mathcal{L}_{\mathcal{B}}, s_l; \eta, \dots) \mapsto (\theta_l^{\text{new}}, s_l^{\text{new}}),$$

where  $s_l$  is the auxiliary state associated with  $\theta_l$ , possibly empty ( $s_l = \emptyset$  for SGD), and  $(\eta, \dots)$  are rule-level hyperparameters. Thus, for the optimizer family considered here, the state decomposes as

$$s = (s_1, \dots, s_L),$$

in parallel with  $\theta$ . No additional cross-layer access is required: the per-layer access established by C2 is sufficient to apply the update rule and maintain its state.

The derivation is now complete. Starting from the three operational pillars, I have shown how I obtained four abstractions: Module, Tensor, Loss, and Optimizer.

Chapter 2 investigates how these abstract requirements can be realized within Rust’s type system and ownership model

## 2 Rust Implementation

| Abstraction      | Cumulative responsibilities                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Module</b>    | <ul style="list-style-type: none"> <li>• Represent and allows the execution of an individual parameterized transformation <math>g_l(z_{l-1}; \theta_l)</math> as an element of an arbitrarily long composition;</li> <li>• provide, under the adopted <i>define-and-run</i> strategy, an ordered, inspectable representation of the composition that is traversable in reverse and maintains the association between each transformation <math>g_l</math>, its parameter block <math>\theta_l</math>, and the preceding computation;</li> <li>• during the forward pass, retain the intermediate values <math>z_l</math> required by the corresponding backward step;</li> <li>• implement internally the VJP of its own transformation,</li> </ul> $\text{VJP}_{g_l} : (\theta_l, z_{l-1}, a_l) \mapsto (a_{l-1}, \nabla_{\theta_l} \ell),$ <p>thereby computing both the local parameter gradient and the adjoint to be propagated to the preceding step;</p> <ul style="list-style-type: none"> <li>• expose the resulting <math>(\theta_l, \nabla_{\theta_l} \ell)</math> pairs to the external optimization mechanism, without applying parameter updates itself.</li> </ul> |
| <b>Tensor</b>    | Provide a representation for the numerical values exchanged between successive transformations, such that the output of each transformation is admissible as the input of the next one.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Loss</b>      | <p>Provide a dedicated abstraction, external to the chain of Modules, for the terminal objective evaluation. Given the final network output <math>z_L</math> and the target <math>y</math>, compute both the scalar sample-wise loss</p> $\ell(z_L, y) \in \mathbb{R}_{\geq 0}$ <p>and the seed of the reverse accumulation,</p> $a_L = \nabla_{z_L} \ell(z_L, y).$ <p>The produced adjoint must have the same shape as <math>a_l</math> for each admissible <math>l</math>, so that it is a valid input adjoint for the VJP of the final Module.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Optimizer</b> | <ul style="list-style-type: none"> <li>• Consume the per-example parameter gradients exposed by the Module mechanism and accumulate them, if necessary, across the examples of a batch to obtain the batch gradient</li> </ul> $\nabla_{\theta_l} \mathcal{L}_{\mathcal{B}} = \frac{1}{ \mathcal{B} } \sum_{i \in \mathcal{B}} \nabla_{\theta_l} \ell_i$ <p>for every parameter block <math>\theta_l</math>;</p> <ul style="list-style-type: none"> <li>• It has to be possible to access to <math>(\theta_l, \nabla_{\theta_l} \mathcal{L}_{\mathcal{B}})</math> for <math>l = 1, \dots, L</math></li> <li>• apply a parameterized update rule to each parameter block and maintain any auxiliary state required by that rule, with the state decomposed as <math>s = (s_1, \dots, s_L)</math> and with <math>s_l = \emptyset</math> for stateless methods such as SGD.</li> </ul>                                                                                                                                                                                                                                                                                               |

Table 1: Cumulative responsibilities of the four abstractions derived from the structural, differentiation, and optimization requirements.

## 2.1 Design of the Module Abstraction

In order to implement the `Module` abstraction, we must first translate into code the semantic requirements established in the previous chapter.

Building on these, our initial formulation of composition required nothing more than the existence of a finite mechanism, not indexed by  $L$ : an abstract constraint that, as we noted at that point, did not in itself impose a single, uniform interface for each transformation. However, the progressive introduction of additional semantic requirements, together with the extensions they necessitated, substantially refines this original formulation.

The mechanism must guarantee an ordered, inspectable, and backward-traversable representation of the composition, maintaining the univocal association between the transformation  $g_l$  and its parameters  $\theta_l$ . To this must be added the need to retain, during the forward pass, the intermediate values  $z_l$  required by the corresponding backward step, the internal implementation of the VJP, and the re-exposure of  $\theta_l$  and  $\nabla_{\theta_l} \ell$ . Taken together, these requirements turn semantic uniformity from a mere design choice into a necessary architectural constraint.

Without a single operational contract, in fact, both the execution of the backward pass, in propagating the gradient  $a_l$  through the various transformations, and the optimizer's update cycle would be forced to handle ad hoc cases for each specific layer type. This would introduce a branching of checks that would destroy the linearity of the chain, increasing code complexity and defeating the very idea of generic composition.

It is precisely from this necessity that the challenge shifts to the implementation level: this semantic uniformity must now be translated concretely into Rust's type system. This brings us to the central question of the present section: *how can we guarantee, in Rust, the semantic uniformity of the interface over a collection of elements that are physically heterogeneous in memory?*

### 2.1.1 Polymorphism as a Foundation for Interface Uniformity

In order to translate the notion of semantic uniformity into an operational construct, we rely on a concept introduced during the course: *polymorphism*.

As we have seen in class, the standard taxonomy distinguishes two broad families: *ad hoc polymorphism* and *universal polymorphism*.

Ad hoc polymorphism comprises two principal forms:

- **Overriding**: a mechanism whereby a subclass redefines a method inherited from a superclass, with the specific implementation selected at runtime via dynamic binding based on the receiver's dynamic type. Within the type system hierarchy, overriding introduces *ad hoc* polymorphism within the context of inheritance-based universal polymorphism. Since this mechanism is intrinsically linked to class-based inheritance, languages lacking traditional object-oriented inheritance, such as Rust, do not implement this form of polymorphism.
- **Overloading**, an ad hoc polymorphism mechanism whereby multiple functions or operators share the same name but have different signatures. The correct implementation is selected on the basis of the static types of the arguments, resulting in early (static) binding in statically typed languages. While classical overloading is common in languages like Java and C++, Haskell and its descendant Rust handle these concepts cleanly through type classes and traits rather than traditional ad hoc function overloading.

Neither of these two forms, however, is what is required here.

What we need is *universal polymorphism*: the ability to provide a single, universal solution where only one algorithm applies to objects of different types. Unlike ad hoc polymorphism, where the same function name denotes different algorithms determined by the actual types, universal polymorphism allows a single piece of code to operate uniformly across multiple types without needing specialized implementations for each.

Universal polymorphism itself admits two realizations, *parametric polymorphism* and *inclusion polymorphism*, which we examine below as candidate mechanisms for composing layers.

### 2.1.2 The Module Trait

The natural vehicle for expressing this uniformity in Rust is the `trait` construct. As we have seen in class, a trait can be understood as a formal contract: a named set of method signatures that a type must implement in order to be considered a member of the trait, without the contract itself prescribing anything about how the implementing type is laid out in memory or allocated.

Under this view, every layer of the network is required to provide an implementation of a common `Module` trait.

Refining our provisional design to incorporate ownership transfer for activations and the required gradient reset mechanism, the definitive formulation of this trait is the following:

```
1 pub trait Module {  
2     /// Computes the output tensor  $z_l$  from the input tensor  $z_{l-1}$  via ownership  
    transfer.
```

```

3     fn forward(&mut self, input: Tensor) -> Tensor;
4
5     /// Given the upstream gradient  $a_l = d \text{ loss} / d z_l$ ,
6     /// returns the downstream gradient  $a_{l-1} = d \text{ loss} / d z_{l-1}$ .
7     fn backward(&mut self, grad_output: Tensor) -> Tensor;
8
9     /// Exposes read-only access to the layer's parameters  $\theta_l$ .
10    fn params(&self) -> Vec<&Tensor>;
11
12    /// Exposes read-write access to the layer's parameters  $\theta_l$ .
13    fn params_mut(&mut self) -> Vec<&mut Tensor>;
14
15    /// Exposes the gradients w.r.t. the parameters, grad  $\theta_l$  L.
16    fn grads(&self) -> Vec<&Tensor>;
17
18    /// Resets all accumulated gradient buffers to zero.
19    fn zero_grad(&mut self);
20 }

```

Here `Tensor` denotes our heap-backed, reference-counted data structure.

The signature of `forward` reflects the requirement that a transformation  $g_l$  take ownership of an input tensor and map it to an output tensor; `backward` reflects the requirement that the layer be able to propagate a gradient  $a_l$  backward into  $a_{l-1}$ , using whatever intermediate state  $z_l$  it retained during the forward pass; `params`, `params_mut`, and `grads` reflect the requirement to re-expose  $\theta_l$ ,  $\nabla_{\theta_l} \ell$ , and allow parameter updates to the optimizer.

### 2.1.3 The Problem of Heterogeneous Composition

Once the `Module` trait has been defined, if only informally, the next question is:

*how a sequence of layers is to be composed.*

Naively, one might attempt to store all layers in a single, homogeneous collection such as `Vec<Module>`. This is, however, not directly possible: `Module` is a trait, not a concrete type, and a `Vec<T>` requires `T` to be a single, statically known type of fixed size. Each concrete struct implementing `Module` (a linear layer, an activation, and so on) will in general have a different size and internal layout, so no single `T` can represent all of them. We are therefore confronted with a genuine requirement for *heterogeneity*: the collection must be able to hold values of different concrete types, all satisfying the same interface.

Two mechanisms are available in Rust to address this requirement, corresponding to the two forms of universal polymorphism introduced above.

### 2.1.4 Parametric Polymorphism: Composition via Generics

The first approach is to use *parametric polymorphism*, i.e., Rust's generic type parameters, to build composition structurally rather than through a homogeneous collection. Two modules can be chained together by means of a generic wrapper struct:

```

1 pub struct Chain<A, B> {
2     pub first: A,
3     pub second: B,
4 }
5
6 impl<A: Module, B: Module> Module for Chain<A, B> {
7     fn forward(&mut self, input: Tensor) -> Tensor {
8         let z = self.first.forward(input);
9         self.second.forward(z)
10    }
11
12    fn backward(&mut self, grad_output: Tensor) -> Tensor {
13        let a = self.second.backward(grad_output);
14        self.first.backward(a)
15    }
16
17    fn params(&self) -> Vec<&Tensor> {
18        let mut p = self.first.params();
19        p.extend(self.second.params());
20        p
21    }
22 }

```

```

23     fn params_mut(&mut self) -> Vec<&mut Tensor> {
24         let mut p = self.first.params_mut();
25         p.extend(self.second.params_mut());
26         p
27     }
28
29     fn grads(&self) -> Vec<&Tensor> {
30         let mut g = self.first.grads();
31         g.extend(self.second.grads());
32         g
33     }
34
35     fn zero_grad(&mut self) {
36         self.first.zero_grad();
37         self.second.zero_grad();
38     }
39 }

```

The key observation is that, whenever A and B both implement `Module`, `Chain<A, B>` itself implements `Module`; the construction is therefore closed under composition, and arbitrarily long chains can in principle be built by nesting, e.g., `Chain<A, Chain<B, C>>`.

The advantages of this approach follow directly from the fact that Rust generics are a *zero-cost abstraction*: at compile time, the compiler *monomorphizes* each distinct instantiation of `Chain`, generating specialized, statically dispatched code with no run-time indirection.

This makes aggressive inlining of `forward` and `backward` across layer boundaries possible, and, since the size and layout of every `Chain<A, B>` instance is known at compile time, the whole composed structure can be allocated contiguously (e.g., on the stack), which in turn improves data locality and cache behavior relative to a heap-allocated, pointer-chasing representation.

The central limitation, however, is that the *type* of the composed network encodes its entire structure: a network of  $L$  layers has a type of nesting depth  $L$ , e.g., `Chain<A, Chain<B, Chain<C, D>>>`. This has two undesirable consequences.

First, the type quickly becomes unwieldy and effectively prevents building networks whose depth or structure is determined at run time (e.g., read from a configuration file), since the type of the resulting composition would have to be known at compile time.

Second, and more importantly for the architecture as a whole, abstractions that operate over the entire network become coupled to its exact nested type. Consider a simple `Optimizer`, the implementation of which will be discussed in subsequent sections: although it depends exclusively on the `Module` interface, it must nonetheless be generic with respect to the model's concrete type:

```

1  pub trait Optimizer<M: Module> {
2      fn step(&mut self, model: &mut M);
3  }
4
5  pub struct Sgd<M: Module> {
6      // fields omitted, not yet specified
7  }
8
9  impl<M: Module> Optimizer<M> for Sgd<M> {
10     fn step(&mut self, model: &mut M) { /* ... */ }
11 }
12
13 // The full nested type must still be spelled out here:
14 let model: Chain<Linear, Chain<ReLU, Chain<Linear, Softmax>>> = /* ... */;
15
16 let mut opt: Sgd<Chain<Linear, Chain<ReLU, Chain<Linear, Softmax>>>> = /* ... */;

```

Thus, adding, removing, or reordering a single layer changes the model type and consequently the type of every abstraction parameterized by it, such as the optimizer, rather than allowing them to depend solely on the `Module` interface.

Moreover, while monomorphization does yield measurable performance benefits, in the setting of deep learning the actual computational bottleneck lies elsewhere, namely in the dense matrix operations performed inside each layer, so the zero-cost guarantee offered by this approach addresses a cost that is not the dominant one in practice.

### 2.1.5 Inclusion Polymorphism: Composition via Trait Objects

The second approach to heterogeneous `Module` implementations is *inclusion polymorphism* via Rust *trait objects*. Unlike statically resolved generics like `Chain<A, B>`, a trait object of type `dyn Module` erases the concrete type, deferring method resolution to runtime.

Because Rust requires every stack variable to have a statically known size, and a `dyn Module` has an unknown, variable size (e.g., a `Linear` layer takes 200 bytes while a `ReLU` takes 0), it is classified as a *dynamically sized type* (DST). Consequently, a bare declaration like `let x: dyn Module` is rejected by the compiler.

To bypass this stack limitation, the concrete value is allocated on the heap while its size is still known, leaving only a fixed-size reference on the stack. This is precisely what `Box<dyn Module>` provides: a stack-resident handle owning a heap-allocated value. This alone, however, only solves the *size* problem, not the *dispatch* problem: a raw data pointer is insufficient because a runtime call like `layer.forward(x)` still cannot tell, from the pointer alone, which underlying type, and hence which implementation of `forward`, it is addressing.

To solve this, Rust employs a *fat pointer* spanning 16 bytes on 64-bit architectures, combining two elements:

1. A **data pointer** pointing to the heap-allocated object.
2. A **vtable pointer** referencing a static, precompiled table of function pointers for that specific concrete type.

This uniform 16-byte representation enables heterogeneous collections such as `Vec<Box<dyn Module>>`, where elements vary in type and size but share an identical stack footprint.

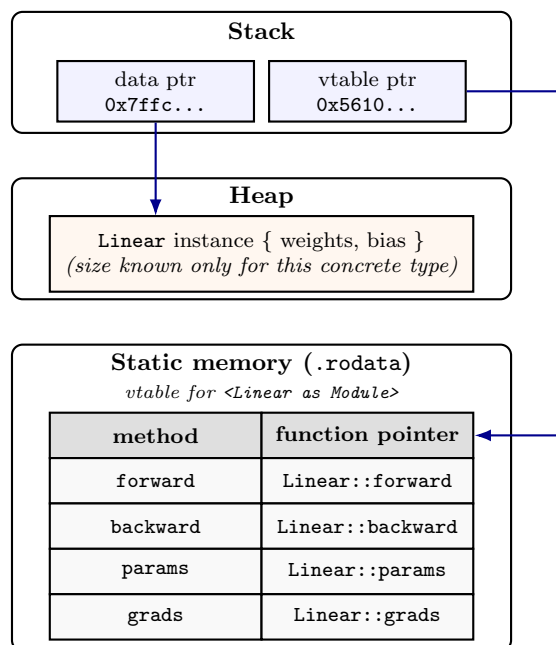


Figure 3: Runtime layout of a `Box<dyn Module>` holding a `Linear` value, read top to bottom. The *stack* holds only the fat pointer itself: two machine words of fixed total size (16 bytes), regardless of the concrete type behind them. The *data pointer* addresses the actual `Linear` instance on the *heap*, whose size is known only because the concrete type is still visible at the point of allocation. The *vtable pointer* addresses a fixed-size table in *static memory*, generated once per concrete type at compile time, mapping each method of `Module` to the concrete function implementing it for `Linear`. A call to `layer.forward(x)` follows the vtable pointer, looks up the `forward` row, and invokes the function pointer found there with the data pointer supplied as the receiver (`self`).

There is a detail we have quietly skipped over: not every trait can be turned into a trait object. The vtable mechanism, as just described, only works if the trait’s methods satisfy two conditions. `Module` happens to satisfy both, which is exactly why `Box<dyn Module>` was usable above without any complication, but it is worth seeing why, since the construction would fail to compile otherwise.

**First constraint: no method may introduce an extra generic type parameter.** A single vtable entry is one function pointer to one piece of compiled code, but a generic method compiles to a *different* piece of code for every type it is called with (monomorphization). In principle, if the compiler saw the entire program at once, it could just count the used types and build a custom vtable. In reality, imagine publishing a library: if someone uses it tomorrow with a brand-new type, the compiler cannot retroactively add a new slot to your vtable. One might argue: “*Just recompile the library from source!*” But if Rust rebuilt vttables based on global usage, a trait’s memory layout and validity would constantly shift depending on function calls written thousands of lines away. Rust strictly enforces



local reasoning: a trait’s contract must be predictable and fixed from its definition alone. Therefore, reserving a finite, stable number of slots for a generic method is impossible.

```
1 trait Module {
2     fn load_into<T>(&self, target: &mut T); // rejected: dyn Module is illegal
3     fn forward(&self, x: Tensor) -> Tensor; // fine: no type parameters
4 }
```

Module, as actually written, has no method like `load_into`: `forward`, `backward`, `params`, `grads` each compile to exactly one function per concrete type, i.e. exactly one vtable entry.

**Second constraint: no method may take or return Self by value.** Suppose Module included a method like `fn duplicate(&self) -> Self`. In principle, returning an object by value simply means copying its bytes into a reserved spot on the caller’s stack. But imagine writing a function that relies on dynamic dispatch:

```
1 fn clone_layer(m: &dyn Module) {
2     let copy = m.duplicate(); // How much memory do we allocate for 'copy'?
3 }
```

Here is the physical roadblock: the compiler must reserve stack space for `copy` *before* the program runs. If the hidden type behind `m` is a `ReLU`, it needs 0 bytes; if it is a complex `Linear` layer, it might need 200 bytes. Because `dyn Module` intentionally erases the original type, the compiler is entirely blind to the concrete struct and cannot possibly guess the required size. Module, as written, avoids this trap: its methods only take pointers (`&self` or `&mut self`, which are always the same size) and return fixed-size types like `Tensor`, guaranteeing that the memory layout is universally predictable. Because Module satisfies both constraints, `dyn Module` is a legal type, and the collection built above is valid: a common pointer type can store any concrete struct implementing Module, regardless of its size or layout.

**Sequential** Having established that `dyn Module` is a legal trait object, we introduce a concrete type built around `Vec<Box<dyn Module>>`, called `Sequential`. Its role is twofold: own a run-time extensible, heterogeneous list of layers, and itself satisfy the Module contract, so that an entire network can be handled from the outside exactly like a single layer, nested inside another `Sequential`, or passed to an `Optimizer` written solely against the Module interface.

```
1 pub struct Sequential {
2     layers: Vec<Box<dyn Module>>,
3 }
4
5 impl Sequential {
6     pub fn new() -> Self {
7         Self { layers: Vec::new() }
8     }
9
10    pub fn add(&mut self, layer: Box<dyn Module>) -> &mut Self {
11        self.layers.push(layer);
12        self
13    }
14 }
```

At this stage `Sequential` is only a growable box of trait objects; it does not yet behave as a Module. We establish conformance by explicitly implementing its methods, with each method walking through `self.layers`:

```
1 impl Module for Sequential {
2     fn forward(&mut self, input: Tensor) -> Tensor {
3         let mut current = input;
4         for layer in self.layers.iter_mut() {
5             current = layer.forward(current);
6         }
7         current
8     }
9
10    fn backward(&mut self, grad_output: Tensor) -> Tensor {
11        let mut current = grad_output;
12        for layer in self.layers.iter_mut().rev() {
13            current = layer.backward(current);
14        }
15        current
16    }
17 }
```

```

17
18     fn params(&self) -> Vec<&Tensor> {
19         self.layers.iter().flat_map(|l| l.params()).collect()
20     }
21
22     fn params_mut(&mut self) -> Vec<&mut Tensor> {
23         self.layers.iter_mut().flat_map(|l| l.params_mut()).collect()
24     }
25
26     fn grads(&self) -> Vec<&Tensor> {
27         self.layers.iter().flat_map(|l| l.grads()).collect()
28     }
29
30     fn zero_grad(&mut self) {
31         for layer in self.layers.iter_mut() {
32             layer.zero_grad();
33         }
34     }
35 }

```

**forward** folds the input through the layers in **add** order via ownership transfer, feeding  $z_l$  as the input to  $g_{l+1}$ ; **backward** performs the symmetric traversal via **rev()**, propagating  $a_l$  from the last layer to the first. Both methods accept tensors by value (**Tensor**) in accordance with the refined ownership model of the **Module** trait, allowing tensors to flow seamlessly through the chain via **move** semantics. **params** and **grads** use **flat\_map** rather than **map** because each **l.params()** already returns a **Vec<&Tensor>**: this flattens the per-layer vectors into one single **Vec<&Tensor>** for the whole network, instead of a nested **Vec<Vec<&Tensor>>**, re-exposing every  $(\theta_l, \nabla_{\theta_l} \ell)$  pair to the optimizer regardless of layer type.

With this construction, the design of the **Module** abstraction is complete at the level of interface uniformity: heterogeneous layers are composed into a single, ordered, traversable structure, without sacrificing the semantic contract established at the beginning of this section.

## 2.2 Design of the Tensor Abstraction

Table 1 assigns to **Tensor** a single, apparently modest responsibility: to represent the values exchanged between successive transformations so that the output of  $g_l$  is admissible as the input of  $g_{l+1}$ . Read literally, this is a statement about the *forward* pass only: it guarantees that  $z_l$  can be fed into  $g_{l+1}$ . Before turning to how **Tensor** should be represented in Rust, it is worth asking whether this same guarantee extends, for free or otherwise, to the *backward* pass, that is, whether the adjoint  $a_{l-1}$  produced by  $\text{VJP}_{g_l}$  is automatically admissible wherever  $z_{l-1}$  was. If it is, a single **Tensor** type suffices for both directions of data flow; if it is not, the design needs a second, dedicated type for adjoints, and every signature in **Module** would have to be duplicated accordingly.

### 2.2.1 Dimensional Compatibility Along the Chain

**Claim.** For every admissible  $l$ , the adjoint  $a_l$  has exactly the same dimension as  $z_l$ .

The backward recursion already gives, for  $l = L, \dots, 1$ ,

$$a_{l-1} = (\partial g_l / \partial z_{l-1})^\top a_l.$$

Write  $J_l := \partial g_l / \partial z_{l-1} \in \mathbb{R}^{n_l \times n_{l-1}}$ , where  $n_{l-1}, n_l$  denote the dimensions of  $z_{l-1}, z_l$  respectively — so  $J_l$ ’s shape is fixed purely by the two vectors it maps between, whatever  $g_l$  is. Transposing swaps its two dimensions,  $J_l^\top \in \mathbb{R}^{n_{l-1} \times n_l}$ , and then

$$a_{l-1} = \underbrace{J_l^\top}_{n_{l-1} \times n_l} \underbrace{a_l}_{n_l} \in \underbrace{\mathbb{R}^{n_{l-1}}}_{\text{same dimension as } z_{l-1}}.$$

This holds for every  $l$  regardless of what  $g_l$  actually computes: only the dimensions of  $J_l^\top$  matter, not its entries, proving the claim.

Since  $a_l$  and  $z_l$  always live in the same space, a single **Tensor** type can carry both, licensing the dual signatures of **Module**:

```
1 fn forward(&mut self, input: &Tensor) -> Tensor;
2 fn backward(&mut self, grad_output: &Tensor) -> Tensor;
```

A dedicated **Adjoint/Gradient** type would add no expressiveness, only duplicate machinery **Tensor** already provides.

### 2.2.2 Representing a Tensor in Rust

With the shape argument settled, **Tensor** can be given a concrete Rust representation.

Following standard practice in dense numerical computing (BLAS-style libraries, NumPy, `ndarray`), the data itself is stored as a single *flat buffer*: one contiguous run of homogeneous scalars, with no structure of its own. All multi-dimensional structure, how many axes there are, how large each is, how to walk from one logical position to the next in memory, is layered *on top of* this buffer as separate metadata, rather than being baked into the storage itself. This separation of “raw storage” from “logical view” is not incidental; it is exactly what will make several operations essentially free in Section 2.2.6.

### 2.2.3 The data Field

A first candidate for the buffer’s type is Rust’s fixed-size array, `[f32; N]`, whose main attraction is that  $N$  is known at compile time: the whole tensor could then be allocated contiguously on the stack, with no heap indirection and a memory layout fully known to the compiler ahead of time, the same kind of benefit that made monomorphized **Chain** $\langle A, B \rangle$  attractive in Section 2.1.

This benefit, however, cannot be realized here, for two independent reasons, each sufficient on its own to rule the array out:

- **The per-example dimensionality depends on the dataset.** An MNIST image contributes  $28 \times 28 = 784$  scalars; another dataset contributes some other count. Baking this into the type as a `const generic`, **Tensor** $\langle \text{const } N: \text{usize} \rangle$ , would make **Tensor** $\langle 784 \rangle$  and **Tensor** $\langle 256 \rangle$  mutually incompatible types, the same “type encodes structure” problem already diagnosed for **Chain** $\langle A, \text{Chain} \langle B, C \rangle \rangle$ , undoing the interface uniformity that motivated **dyn Module** in the first place.
- **The batch size is a run-time quantity.** A **Tensor** does not hold one example but a *batch* of  $B$  examples, and  $B$  is chosen by the training loop, not by the definition of the layer. Worse,  $B$  is not even constant *within a single run*: whenever the dataset size is not an exact multiple of the configured batch size, the last batch of an epoch is smaller than the others. A type whose size is fixed once and for all at compile time cannot represent two different concrete sizes at different points of the same program, let alone a size chosen at run time in the first place.

Both obstacles disappear once the buffer is backed by a heap-allocated `Vec<f32>`: its length is a run-time value, so a single `Tensor` type can hold any per-example dimensionality and any batch size, including one that changes from call to call.

However, a raw `Vec<f32>` imposes strict *exclusive ownership*: the buffer can have only one owner at a time. This becomes a structural roadblock when creating zero-cost views of a tensor, such as its transpose. If `data` were exclusively owned, exposing a transposed `Tensor` while keeping the original intact (e.g., preserving `self.weight` for subsequent forward passes) would force an impossible choice:

- **Clone the buffer:** resulting in an  $O(n)$  physical copy of the underlying floats, which strictly violates the requirement for  $O(1)$  operations on views.
- **Move the buffer:** which would transfer ownership into the new transposed tensor, invalidating the original layer weights and breaking the network’s state.

To achieve “same buffer, zero copies” alongside “multiple independent and usable `Tensors`”, we must introduce *shared ownership*. By wrapping the buffer in a Reference Counted pointer (`Rc<Vec<f32>>`), multiple `Tensor` instances can safely point to the same physical heap allocation. Creating a view now only requires incrementing an integer counter, representing the minimal mechanism Rust offers to share data without copying and without relying on `unsafe` blocks.

```
1 use std::rc::Rc;
2
3 pub struct Tensor {
4     data: Rc<Vec<f32>>, // shared ownership of the flat buffer
5     shape: Vec<usize>, // see Section 3.4.4 below
6     strides: Vec<usize>, // see Section 3.4.5 below
7 }
```

### 2.2.4 The shape Field

On its own, `data` is a meaningless flat run of  $n$  scalars: nothing in a `Vec<f32>` of length  $B \cdot 784$  says whether it holds  $B$  images of 784 pixels each, 784 signals of  $B$  samples each, or something else entirely. The `shape` field, `shape: Vec<usize>`, resolves this ambiguity: it is an ordered list of extents, one per axis, that tells the buffer how to be grouped and nested, equivalently, it fixes the rank of the tensor (its length) and, via  $n = \prod_k \text{shape}[k]$ , the total element count that `data` must have.

For a mini-batch of  $B$  MNIST images kept in their native two-dimensional form, `shape` =  $[B, 28, 28]$ : the buffer is interpreted as  $B$  consecutive blocks of 28 rows of 28 pixels each, with `data.len()` =  $B \cdot 28 \cdot 28$ . If instead the same batch is flattened for a fully-connected input layer, `shape` =  $[B, 784]$  describes the identical buffer, reinterpreted as  $B$  rows of 784 features, same  $n$ , same underlying bytes, different logical grouping. This is exactly the “interpretation layer” role anticipated above.

The reasoning of Section 2.2.3 applies verbatim to `shape` itself: both its length (the rank, which differs between a fully-connected tensor of shape  $[B, 784]$  and a convolutional one of shape  $[B, C, H, W]$ ) and its entries (starting with  $B$ ) are run-time quantities, so `shape` must equally be a `Vec<usize>` rather than a fixed-size array.

### 2.2.5 The shape Field

On its own, `data` is a meaningless flat run of  $n$  scalars: nothing in a `Vec<f32>` of length  $B \cdot 784$  says whether it holds  $B$  images of 784 pixels each, 784 signals of  $B$  samples each, or something else entirely. The `shape` field, `shape: Vec<usize>`, resolves this ambiguity: it is an ordered list of extents, one per axis, that tells the buffer how to be grouped and nested, equivalently, it fixes the rank of the tensor (its length) and, via  $n = \prod_k \text{shape}[k]$ , the total element count that `data` must have.

For a mini-batch of  $B$  MNIST images kept in their native two-dimensional form, `shape` =  $[B, 28, 28]$ : the buffer is interpreted as  $B$  consecutive blocks of 28 rows of 28 pixels each, with `data.len()` =  $B \cdot 28 \cdot 28$ . If instead the same batch is flattened for a fully-connected input layer, `shape` =  $[B, 784]$  describes the identical buffer, reinterpreted as  $B$  rows of 784 features, same  $n$ , same underlying bytes, different logical grouping. This is exactly the “interpretation layer” role anticipated above.

The reasoning of Section 2.2.3 applies verbatim to `shape` itself: both its length (the rank, which differs between a fully-connected tensor of shape  $[B, 784]$  and a convolutional one of shape  $[B, C, H, W]$ ) and its entries (starting with  $B$ ) are run-time quantities, so `shape` must equally be a `Vec<usize>` rather than a fixed-size array.

### 2.2.6 The strides Field

The remaining piece of metadata is `strides`. Given a tensor’s logical `shape` (for example, a  $3 \times 4$  matrix), the `strides` tell us exactly how many positions we need to skip in the underlying flat, one-dimensional memory buffer to move by one step along a specific logical axis (e.g., moving to the next row or the next column).

Because the offset formula treats **shape** and **strides** as independent inputs, any operation that only rearranges *which* multi-index maps to *which* offset, permuting axes, slicing a sub-range, reversing an axis, can be realized by rewriting these two integer tuples alone, in  $O(1)$ , with the underlying buffer untouched: the framework only changes the “reading rule”, never the data. Operations that actually combine values, like elementwise arithmetic or matrix multiplication, still require a full pass over the buffer, but the compute kernel reads it correctly regardless of how it has been reshaped, because it consults **strides** rather than assuming a fixed layout.

For instance, transposing a matrix means defining a new tensor  $T$  with  $T[j, i] := X[i, j]$  for every admissible  $(i, j)$ . This is achieved, with zero data movement, by swapping the two entries of **shape** *and*, correspondingly, the two entries of **strides**. Paired with our `Rc<Vec<f32>>` storage model, the transposition method `.t()` executes in strict  $O(1)$  time and space:

```

1  impl Tensor {
2      pub fn t(&self) -> Tensor {
3          Tensor {
4              data: Rc::clone(&self.data), // O(1): bumps ref count, no float copy
5              shape: self.shape.iter().cloned().rev().collect(), // O(1): small Vec<usize>
6              strides: self.strides.iter().cloned().rev().collect(), // O(1): small Vec<
                           usize>
7          }
8      }
9  }

```

#### Shared Physical Buffer (RAM) — indices 0..5

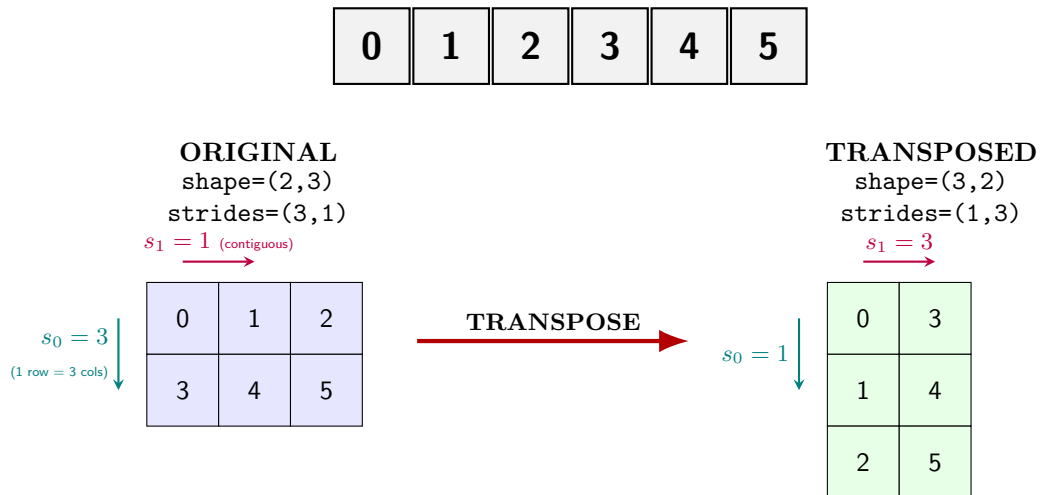


Figure 4: Same physical buffer, two different readings. On the left, advancing by one row (axis 0) skips 3 positions because each row holds 3 columns ( $s_0 = 3$ ); advancing by one column ( $s_1$ ) costs only 1 because elements within a row are contiguous. Transposing swaps **shape** and **strides**: for instance, the same buffer cell holding value 1 appears in a different position in the transposed view (row 1, column 0), with no physical data movement at all.

#### 2.2.7 Ownership of Tensor Across forward, backward, and the Parameter Accessors

Module exposes four points at which a **Tensor** crosses a function boundary: the input of **forward**, the input of **backward**, and the two accessors **params** and **grads**.

**forward: Ownership Transfer as a Structural Necessity** In our architecture, a **Module** is responsible for saving its input during the forward pass so it can be used later in the backward pass. This creates a strict memory lifecycle requirement that makes passing **Tensor** by value the natural and most efficient design choice for this architecture.

If **forward** took its input by reference (`&Tensor`), the layer would need that reference to remain valid until a later call to **backward**. Since the activation is owned by the caller and typically discarded as execution progresses through the network, such a borrow cannot be retained safely. To preserve the activation, the layer would be forced to create its own owned handle:

```

1  // Bad: Borrowing forces a clone, bumping Rc count > 1 and risking CoW overhead later.
2  fn forward(&mut self, input: &Tensor) -> Tensor {
3      self.input_cache = Some(input.clone()); // Rc reference count becomes 2

```

```

4     input.dot(&self.weight.t()) + &self.bias
5 }

```

Taking `input: Tensor` by value solves this entirely. Along a linear chain, ownership of an activation naturally flows from one layer to the next. By transferring ownership (`move`), the layer takes full control of the tensor and caches it while maintaining an exclusive `Rc` handle (`ref_count = 1`). This guarantees that any subsequent in-place operation can modify memory directly without triggering Copy-on-Write (CoW) allocation overhead.

```

1 // Good: Moving ownership preserves exclusive handle and allows zero-cost caching.
2 fn forward(&mut self, input: Tensor) -> Tensor {
3     self.input_cache = Some(input); // Ownership transferred, ref_count
4     // = 1
5     let cached = self.input_cache.as_ref().unwrap(); // Read back via borrow
6     cached.dot(&self.weight.t()) + &self.bias
7 }

```

Crucially, one must distinguish between two levels of shared ownership in this design:

- **Internal Shared Storage (`Rc<Vec<f32>>`):** Managed inside `Tensor` to allow zero-cost structural views (such as matrix transposition via `.t()`) over the same raw float buffer.
- **External Topological Sharing (`Rc<Tensor>`):** Required only when handling non-linear network graph topologies (e.g., skip connections or residual layers where a single activation feeds multiple downstream layers).

```

1 // Shared ownership: Necessary only for branching network architectures.
2 fn forward(&mut self, input: Rc<Tensor>) -> Tensor {
3     let out = input.dot(&self.weight.t()) + &self.bias;
4     self.input_cache = Some(Rc::clone(&input)); // Clones the Tensor handle for branching
5     out
6 }

```

While `Rc<Tensor>` handles branching naturally, making it the default everywhere would introduce unnecessary pointer indirection and reference-counting overhead in the common sequential case.

For standard feed-forward chains, plain `move` semantics remain both the simplest and the most efficient choice.

**backward: Ownership as a Design Choice** While `forward` must take ownership to cache its input, `backward` has no such structural requirement. The incoming gradient (`grad_output`) is used during the operation and is not stored inside `self`.

```

1 fn backward(&mut self, grad_output: Tensor) -> Tensor {
2     let input = self.input_cache.as_ref().unwrap(); // z_{l-1}, cached during forward
3
4     // Gradients are accumulated rather than overwritten
5     self.grad_weight += grad_output.t().dot(input);
6     self.grad_bias += grad_output.sum_axis(Axis(0)).insert_axis(Axis(0));
7
8     grad_output.dot(&self.weight) // Last use of grad_output
9 }

```

Notice that the parameter gradients are strictly accumulated (`+=`) rather than overwritten. While the operational details of this design will be formalized in the context of the `Optimizer` abstraction, this additive semantic is a structural necessity: it permits the accumulation of gradients across multiple independent forward-backward passes before a parameter update is enacted. Consequently, this architectural choice implicitly mandates a dedicated mechanism to zero the gradient buffers before starting a new accumulation cycle.

Because `grad_output` does not need to outlive the method call, taking a reference (`&Tensor`) would compile perfectly without forcing memory copies:

```

1 // This borrowing loop type-checks effortlessly:
2 let mut a = seed_from_loss;
3 for layer in layers.iter_mut().rev() {
4     a = layer.backward(&a); // Old 'a' is dropped, no clones forced
5 }

```

Passing `grad_output: Tensor` by value is therefore a deliberate design choice, driven by two key advantages rather than a compiler constraint:

- **API Uniformity:** It maintains a single, consistent ownership paradigm across the entire `Module` trait, avoiding unnecessary asymmetry between `forward` and `backward`.
- **Explicit Consumption Semantics:** It makes explicit that the backward operation consumes its incoming gradient. This is particularly natural in non-linear architectures, where the execution engine may first accumulate gradients from multiple paths into a single tensor before passing that tensor to the corresponding upstream step.

**params and grads: Borrowing Persistent State** Unlike intermediate activations, which flow through the network and are discarded, a layer’s parameters ( $\theta_l$ ) are persistent state. They belong to the `Module` and must remain available across the entire training lifecycle.

This changes how ownership must be handled. An accessor cannot simply move the parameters out of the module: doing so would transfer ownership and leave the module without its own weights and biases. Since `Tensor` is not `Copy`, returning the existing tensors by value would move them.

While cloning could produce owned `Tensor` handles in  $O(1)$  time thanks to `Rc`, doing so would increment the reference count (`ref_count > 1`). This would be detrimental for the optimizer: when mutating parameters in-place ( $\theta_l \leftarrow \theta_l - \eta \nabla_{\theta_l} \mathcal{L}$ ), a shared `Rc` buffer would trigger Copy-on-Write ( $O(n)$  physical memory duplication) at every single training step.

The appropriate abstraction is therefore to **lend** the existing tensors through references. Different operations require different kinds of access, which leads to three accessors:

```

1 fn params(&self) -> Vec<&Tensor> {
2     vec! [&self.weight, &self.bias]
3 }
4
5 fn params_mut(&mut self) -> Vec<&mut Tensor> {
6     vec! [&mut self.weight, &mut self.bias]
7 }
8
9 fn grads(&self) -> Vec<&Tensor> {
10     vec! [&self.grad_weight, &self.grad_bias]
11 }

```

Each accessor corresponds to a different requirement:

- **The Inspection Loan (params):** Returns immutable references (`&Tensor`) for operations such as checkpointing or logging.
- **The Update Loan (params\_mut):** Returns mutable references (`&mut Tensor`) so an optimizer can update parameters in place ( $\theta_l \leftarrow \theta_l - \eta \nabla_{\theta_l} \mathcal{L}$ ).
- **The Gradient Loan (grads):** Returns immutable references to the stored gradients, which the optimizer reads to compute the update.

The distinction between immutable and mutable loans also matters when the optimizer performs an update. Although parameters and gradients are different fields, the methods expose them through the same `Module` object: `grads(&self)` creates a shared borrow of the module, while `params_mut(&mut self)` requires an exclusive borrow. These borrows therefore cannot overlap.

The optimizer solves this by separating the two phases in time.

It first copies the gradients into owned tensors, ending the immutable borrow, and only then requests mutable access to the parameters:

```

1 impl SGD {
2     fn step(&self, module: &mut dyn Module) {
3         // 1. Immutable borrow: copy gradient handles into owned tensors.
4         // Thanks to Rc<Vec<f32>>, g.clone() is a cheap O(1) metadata copy!
5         let grads: Vec<Tensor> = module.grads()
6             .into_iter()
7             .map(|g| g.clone())
8             .collect();
9
10        // 2. Mutable borrow: update parameters in-place
11        for (theta, grad) in module.params_mut().into_iter().zip(grads.iter()) {
12            *theta -= (&grad * self.lr); // In-place update
13        }
14    }
15 }

```



Crucially, because `Tensor` uses `Rc<Vec<f32>>`, the intermediate `g.clone()` call is a cheap  $O(1)$  operation that merely clones the metadata and increments the `Rc` counter, without physically copying float buffers in memory. It creates owned gradient values so that the immutable borrow on `module` ends before the mutable borrow required by `params_mut()` begins, elegantly satisfying Rust's borrow checker at zero runtime performance penalty.



### 2.3 Design of the Loss Abstraction

While tensors flow uniformly through network layers, loss functions introduce a divergence: the prediction  $z_L$  is always a **Tensor**, but the target  $y$  depends on the objective. Classification targets are naturally discrete class indices, whereas regression targets are continuous quantities matching  $z_L$ .

This section assumes a single-device, CPU-only setting where **Tensor** is a dense, **f32**-only container (Section 2.2.2); device placement and integer-typed tensors are orthogonal concerns. Throughout,  $N$  denotes the number of examples in a batch and  $C$  the number of classes, matching the convention used in Figure 5; for a single example the corresponding costs are  $O(1)$  and  $O(C)$  respectively.

Enforcing a single concrete target type creates an architectural compromise:

- **Fixing the target as `&[usize]`** handles classification naturally, but makes regression inexpressible since continuous values lack a valid index encoding.
- **Fixing the target as `&Tensor`** handles regression and can also represent classification targets, but loses the type-level distinction between discrete labels and continuous values. Storing indices as **f32** therefore erases their semantic distinction: the type no longer expresses that a value is a nominal class label rather than an ordinary continuous quantity, potentially permitting inappropriate metric operations (such as interpolation or distance computations), depending on which operations are exposed on **Tensor**. Alternatively, a dense one-hot encoding removes the ambiguity of mapping a class to a single scalar index, but the target is still statically just a **Tensor** of **f32**, so the type itself does not communicate that it represents a one-hot classification target rather than an arbitrary continuous quantity; it also increases the target representation from  $O(N)$  to  $O(N \times C)$ , introducing unnecessary memory and bandwidth overhead.

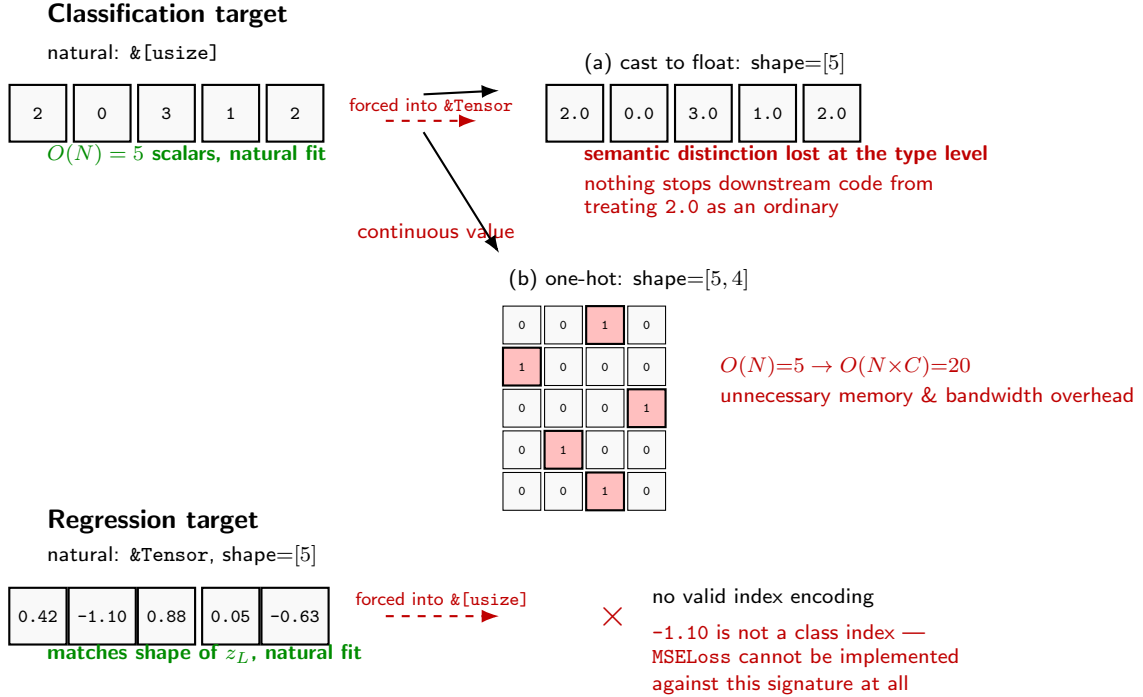


Figure 5: Forcing a single concrete target type produces asymmetric failure modes, with  $N$  examples and  $C$  classes. **Top:** fixing the target as `&Tensor` forces classification labels through one of two broken encodings — (a) casting indices to **f32**, which loses the type-level distinction between discrete labels and continuous values and can allow downstream code to misapply metric operations; or (b) a dense one-hot encoding, which grows the target representation from  $O(N)$  to  $O(N \times C)$  and introduces unnecessary memory and bandwidth overhead, without itself making the target type statically distinguishable from an arbitrary **Tensor**. **Bottom:** fixing the target as `&[usize]` makes regression targets inexpressible.

Although the target representation is inherently tied to the semantics of the loss, in principle the target interface could be designed around a single **Tensor** type, given suitable conventions or encoding metadata. However, the choice to adopt differentiated targets connects directly to the polymorphism taxonomy discussed earlier: rather than relying on universal parametric polymorphism, which would require explicitly threading a generic parameter through the entire call chain, we exploit Rust’s trait-based polymorphism, together with **associated types**, which allow each implementation to specify its own corresponding target type. In particular, the caller does not need to propagate **Target** as a separate generic parameter through the call chain; the associated type is fixed by the concrete type

implementing `Loss` rather than being an independent parameter of the trait or a property of any particular runtime value, so the target type remains part of the `Loss` contract itself.

This mechanism is conceptually reminiscent of Haskell’s type classes with associated type families, although the two systems differ in their type-inference and coherence rules. The analogy is useful insofar as both allow an interface to associate a type with a particular implementation, while preserving static safety and architectural clarity.

Each `Loss` implementation declares its target type directly within the trait definition:

```

1 pub trait Loss {
2   type Target: ?Sized;
3   fn forward(&self, z_l: &Tensor, target: &Self::Target) -> (f32, Tensor);
4 }
5
6 // Classification: accepts a slice of discrete class labels
7 impl Loss for SoftmaxCrossEntropy {
8   type Target = [usize];
9
10  fn forward(&self, z_l: &Tensor, target: &Self::Target) -> (f32, Tensor) {
11    // ...
12  }
13
14
15 }
16
17 // Regression: accepts a continuous activation tensor
18 impl Loss for MSELoss {
19   type Target = Tensor;
20
21  fn forward(&self, z_l: &Tensor, target: &Self::Target) -> (f32, Tensor) {
22    // ...
23  }
24
25
26 }

```

The `?Sized` bound allows the associated type to be dynamically sized (like the slice `[usize]`), while supporting sized types like `Tensor`.

The pair returned by `forward` is  $(\ell, \nabla_{z_L} \ell)$ : the scalar loss alongside its gradient with respect to the prediction `z_l`. Crucially, both quantities are computed as a mean over the  $N$  examples in the batch (i.e., inherently scaled by  $1/N$ ). As we will see, delegating this reduction entirely to the `Loss` abstraction guarantees that the constant factor propagates linearly through the backward pass, allowing upstream components to remain completely agnostic to the batch dimension.

Finally, unlike `Module::forward` (Section 2.2.7), which consumes inputs by value to cache them for backpropagation, `Loss::forward` takes arguments by reference. In this design, the loss computes both  $\ell$  and  $\nabla_{z_L} \ell$  in a single call, so no loss-specific forward state needs to be retained for a later backward traversal. Borrowing is therefore the clean, expected choice for this API; a design that instead separated forward evaluation from a later backward pass would need to retain such state, but that is not the case here.

## 2.4 Design of the Optimizer Abstraction

Having settled `Module`, `Tensor`, and `Loss`, we turn to the fourth and last abstraction of Table 1: `Optimizer`. Its listed duties translate into four design questions:

- how much of the model the optimizer is allowed to know
- how the per-example gradient becomes a batch gradient
- how the mathematical update rule becomes code that mutates `Tensor` state in place
- how, if at all, an optimizer remembers anything from one call to the next

to which a fifth, more operational, question must be added: how these calls are sequenced inside a training loop without violating Rust’s ownership rules.

**How much of the model is the optimizer allowed to know?** To answer the first question, the core principle is that the optimizer (such as SGD) does not know and does not need to know the specific type of layer it is modifying, whether it is a linear layer, a convolution, or an activation. To it, only the raw numbers of the weights ( $\theta$ ) and their gradients exist.

The mechanism in Rust achieves this through dynamic polymorphism. Instead of binding itself to a concrete model type, the `step` function accepts a polymorphic trait object, `&mut dyn Module`, which allows the same optimizer struct to accept any object implementing the `Module` trait:

```
1 pub trait Optimizer {  
2     fn step(&mut self, module: &mut dyn Module);  
3 }
```

The practical advantage is that the exact same optimization code works without modification, whether you are training a single isolated layer or an entire, complex, nested `Sequential` network. Through this design, the optimizer is completely decoupled from the underlying architecture, avoiding the rigid coupling that a generic type parameter like `Optimizer<M: Module>` would have introduced.

**How does the per-example gradient become a batch gradient?** Regarding the second question, Table 1 assigns the `Optimizer` the duty of computing the batch gradient:  $\nabla_{\theta_l} \mathcal{L}_B = \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta_l} \ell_i$ . In practice, this reduction is almost entirely handled before the optimizer even accesses the gradients.

First, the **summation** over the batch is a free by-product of matrix algebra. Because the `Tensor` shape includes the batch axis  $B$  (Section 2.2.5), the backward pass computation (e.g., `grad_output.t().dot(input)` in a linear layer) naturally contracts over this axis. Consequently, a single call to `backward` on a batch automatically yields the sum of the gradients, not  $|B|$  separate tensors.

Second, the **division** by  $|B|$  to obtain the mean is delegated to the `Loss` abstraction rather than the optimizer. Because the VJP is a linear operation (Section 2.2.1), scaling the initial backward seed  $a_L = \nabla_{z_L} \ell$  by  $1/|B|$  propagates this exact constant factor unaltered through every subsequent  $\text{VJP}_{g_l}$ . Thus, the gradients exposed by `grads()` are already correctly averaged, allowing the `Optimizer` to remain completely unaware of the batch dimension.

The “if necessary” clause of Table 1 covers the remaining scenario: **gradient accumulation**. If a logical batch  $B$  is too large to fit in memory, it must be split into smaller micro-batches. To accumulate gradients across multiple forward/backward passes before a single optimization `step` is taken, the layer’s `backward` method must add to the existing gradients (e.g., `self.grad_weight += ...`) rather than overwriting them. This design choice strictly necessitates an explicit `zero_grad` pass to clear the accumulated buffers before starting the next batch cycle.

**How does the mathematical update rule become code that mutates `Tensor` state in place?** Addressing the third question, with  $(\theta_l, \nabla_{\theta_l} \mathcal{L}_B)$  available for every  $l$ , translating an update rule such as plain SGD,  $\theta_l \leftarrow \theta_l - \eta \nabla_{\theta_l} \mathcal{L}_B$ , into code is a direct application of the `params_mut/grads` pair of Section 2.2.7:

```
1 pub struct Sgd { lr: f32 }  
2  
3 impl Optimizer for Sgd {  
4     fn step(&mut self, module: &mut dyn Module) {  
5         let grads: Vec<Tensor> = module.grads().into_iter().cloned().collect();  
6         for (theta, grad) in module.params_mut().into_iter().zip(grads.iter()) {  
7             *theta -= &(grad * self.lr); // in-place update  
8         }  
9     }  
10 }
```

As established in Section 2.2.7, the intermediate `.cloned().collect()` is not incidental: it closes the shared borrow opened by `grads(&self)` before the exclusive borrow required by `params_mut(&mut self)` is requested, since the two cannot overlap on the same `Module`. Crucially, thanks to our `Rc<Vec<f32>>` storage, this mandatory borrow-checker workaround incurs zero physical memory duplication, executing as a fast  $O(1)$  metadata clone.

**How does an optimizer remember anything from one call to the next?** To answer the fourth question, we must look at auxiliary state. While standard SGD is stateless and requires only the current weights and gradients ( $s_l = \emptyset$ ), more advanced algorithms like Momentum require an auxiliary state. Specifically, Momentum relies on a velocity term  $v_l \leftarrow \beta v_l + \nabla_{\theta_l} \mathcal{L}_{\mathcal{B}}$  that must be updated at every step and remembered for the next ( $s_l = v_l \neq \emptyset$ ).

Because optimization algorithms and their internal auxiliary states are fundamentally decoupled from the forward and backward mechanics of neural network layers, this historical data cannot reside within the `Module`.

A layer is structurally agnostic to how it is optimized—requiring no auxiliary state for plain SGD, a velocity term for Momentum, or multiple moment buffers for more complex methods like Adam. Consequently, this state must reside within the `Optimizer` itself, stored as an internal, positionally-indexed `Vec` with one velocity tensor per parameter block. Furthermore, since the optimizer does not know the network’s architecture ahead of time, this state is lazily allocated during the very first training step:

```
1 pub struct Momentum {
2     lr: f32,
3     beta: f32,
4     velocity: Vec<Tensor>, // s_l, l = 1..L; empty until first step
5 }
6
7 impl Optimizer for Momentum {
8     fn step(&mut self, module: &mut dyn Module) {
9         let grads: Vec<Tensor> = module.grads().into_iter().cloned().collect();
10
11         // Lazy allocation: initialized only on the first step
12         if self.velocity.is_empty() {
13             self.velocity = grads.iter().map(Tensor::zeros_like).collect();
14         }
15
16         // Zipping parameters, gradients, and velocities by position
17         for ((theta, grad), v) in module.params_mut().into_iter()
18             .zip(grads.iter())
19             .zip(self.velocity.iter_mut())
20         {
21             *v = &(&*v * self.beta) + grad;
22             *theta -= &(&*v * self.lr);
23         }
24     }
25 }
```

Because `zip` matches elements strictly by their index position (pairing the  $i$ -th parameter with the  $i$ -th gradient and  $i$ -th velocity), the update loop is entirely blind to the actual identity or layer-ownership of the tensors. It relies on absolute positional alignment across three independent collections: the mutable parameters returned by `params_mut()`, the cloned gradients in `grads`, and the historical states stored in `self.velocity`.

This introduces a strict operational invariant: `params_mut()` and `grads()` must enumerate the parameter blocks in the exact same deterministic order on every single training step. If a custom module implementation were to alter its traversal order between calls, `zip` would silently mispair updates—applying a velocity tensor belonging to one layer’s weight matrix to an entirely different layer’s bias vector, corrupting training without triggering any runtime panic. While our `Sequential` container (Section 2.1) naturally upholds this invariant by iterating over its layers in a fixed order, the `Module` trait signature cannot mathematically enforce it. It remains a structural convention—a functional contract between `Module` and `Optimizer` that must be respected by the developer, as it is not statically verified by Rust’s type system.

**How are these calls sequenced inside a training loop without violating Rust’s ownership rules?** Finally, to answer the fifth, more operational question, we must consider the training lifecycle. The reduction in Section 2.4 assumed `backward accumulates` into `grad_weight` (`self.grad_weight += ...`) rather than overwriting it as originally shown. This one-word change has a consequence: without an explicit reset, gradients from a previous step would silently add onto the next one. Symmetrically to how `params/grads/params_mut` were justified in Section 2.2.7 by parameters being persistent state *owned* by the module, resetting that same state is the module’s responsibility, not the optimizer’s — `Optimizer` only ever reads gradients, it never has a reason to clear them:

```

1 pub trait Module {
2     // ...forward, backward, params, params_mut, grads as before...
3     fn zero_grad(&mut self);
4 }
5
6 impl Module for Sequential {
7     fn zero_grad(&mut self) {
8         for layer in self.layers.iter_mut() { layer.zero_grad(); }
9     }
10 }

```

The training loop is then a strict sequence of three phases, none of which may overlap in time:

```

1 model.zero_grad();
2 for micro_batch in batch.chunks(k) { // 1..K micro-batches
3     let z_l = model.forward(micro_batch.input);
4     let (_, grad_out) = loss_fn.forward(&z_l, &micro_batch.target);
5     model.backward(grad_out); // accumulates into grad_*
6 }
7 optimizer.step(&mut model); // consumes and updates

```

This strict ordering is not merely good practice: `model.forward/model.backward` and `optimizer.step` each require an exclusive `&mut` borrow of `model`, so the borrow checker itself would reject any attempt to interleave them. The same constraint that, in Section 2.2.7, forced the gradient-then-parameter borrows inside a single `step` to be sequenced rather than simultaneous, here forces the entire lifecycle, reset, accumulate, update, into non-overlapping phases at the scale of the whole training loop.

**Advanced Refinements** Two common refinements confirm, rather than complicate, the decoupling discussed in our first question (Section 2.4): both are implemented entirely inside `Optimizer::step`, with no change whatsoever to `Module` or `Tensor`.

**Weight decay.** L2 regularization modifies the update rule to  $\theta_l \leftarrow \theta_l - \eta(\nabla_{\theta_l} \mathcal{L}_B + \lambda \theta_l)$ . Since `params_mut()` already exposes  $\theta_l$  itself, not only its gradient, the extra term is a purely local addition to `step`:

```

1 for (theta, grad) in module.params_mut().into_iter().zip(grads.iter()) {
2     let decayed = grad + &(&*theta * self.weight_decay);
3     *theta -= &(decayed * self.lr);
4 }

```

**Gradient clipping.** To guard against numerically exploding gradients, the global norm  $\|\nabla \mathcal{L}_B\| = (\sum_l \|\nabla_{\theta_l} \mathcal{L}_B\|^2)^{1/2}$  is computed over the already-owned `grads`: `Vec<Tensor>` local copy — the same clone introduced in Section 2.2.7 to release the borrow on `module`. If it exceeds a threshold, every gradient is rescaled before being consumed by the update rule:

```

1 fn clip_grad_norm(grads: &mut [Tensor], max_norm: f32) {
2     let total_norm = grads.iter().map(Tensor::squared_sum).sum::<f32>().sqrt();
3     if total_norm > max_norm {
4         let scale = max_norm / (total_norm + 1e-6);
5         for g in grads.iter_mut() { *g *= scale; }
6     }
7 }

```

This reveals a profound consequence of our `Rc`-based design. The `grads` collection holds  $O(1)$  clones of the module’s gradients. If the threshold is *not* exceeded, they are consumed by the update rule with zero memory allocation. Only if the threshold *is* exceeded, the in-place mutation (`*g *= scale`) triggers Rust’s `Rc::make_mut` under the hood. Detecting that the reference count is greater than 1 (since the module still owns the original buffers), the `Tensor` gracefully performs a Copy-on-Write, allocating a deep copy specifically for the scaled values without corrupting the module’s internal state. The architecture natively yields optimal lazy allocation: physical memory is duplicated strictly if and when mathematical modification is required.

## 3 How I Used AI

My use of AI in this project falls into two clearly separated phases.

- **Phase One — Design.** Everything up to and including the writing of this document: the architectural reasoning, the mathematical derivations, the figures and the prose. Here I used only the free web interfaces of ChatGPT, Claude and Gemini, and I kept total control.
- **Phase Two — Implementation.** Everything from this document onwards: the translation of the specification into a working Rust codebase. Here I used Claude Code, an agentic tool integrated with my editor, and I deliberately loosened my grip.

The two phases are discussed separately below, as both the way I used the AI and evaluated it, as well as the lessons I learned from it, differ significantly.

### 3.1 Phase One: Designing the System

Throughout the design phase I used various AI tools, specifically the free web interfaces of ChatGPT, Claude and Gemini. I deliberately avoided integrated IDE agents such as Claude Code or Codex.

The guiding principle behind this workflow was absolute ownership: I wanted to retain total control over both the architectural abstractions and the implementation details. Consequently, I treated the AI not as an autonomous developer to whom I could outsource large modules, but rather as an interactive sounding board. My approach was iterative, yet I explicitly avoided the pattern of delegating a massive task and then endlessly tweaking a bloated, AI-generated output.

#### 3.1.1 The Trade-off: Speed versus Comprehension

Reflecting on this process, I came to a stark realisation about the current state of AI. Even using only free-tier models, the AI could probably have completed this entire project in a fraction of the time, had I simply allowed it to take the wheel. Doing so, however, requires a fundamental sacrifice: abandoning total control over complexity, and with it deep personal comprehension.

I am unsure what the future holds for developers, but blindly delegating logic to an AI and then attempting to reverse-engineer its reasoning *post factum* is a massive cognitive burden, one that current models are not yet quite capable of helping us navigate. For repetitive, boilerplate or non-critical tasks, trading control for speed makes sense. But for the core challenges of this project, such as designing a minimal, tightly interconnected library from scratch, delegation was not an option: outsourcing even a single central component would have compromised my understanding of the entire system.

Where I *did* use the AI aggressively was in the mere act of typing. Once I had solidified my ideas and architectural choices, I used the AI as a highly advanced compiler, translating my logic into code.

#### 3.1.2 A Valid Self-Critique

In hindsight, a fair self-critique is that I may have gripped the steering wheel a little too tightly. The future of software engineering probably lies in finding a balanced synergy with these tools. My rigid desire for total control is heavily biased by my professional background: having previously worked in defence software, absolute architectural and implementational certainty was not merely a preference but a strict requirement.

#### 3.1.3 Rapid Voice Prompting

Beyond coding, I frequently use AI to study. During this project I experienced first-hand that seeking deep understanding does not necessarily require the most powerful models, nor massive, comprehensive outputs.

In fact, my most effective learning occurred when I used faster, slightly weaker models through **voice commands** (speech-to-text, not the continuous audio-chat mode). I explicitly instructed the AI to give me extremely brief, punchy answers. This allowed me to read each response almost instantly and fire back with my doubts, creating a fast-paced, Socratic ping-pong. By forcing the model to take small steps with me, I converged on solutions that I genuinely understood.

I believe this worked because the reasoning process remained primarily mine. Rather than passively consuming long explanations, I continuously steered the conversation towards the specific points I wanted to clarify. The AI acted more like an interlocutor than a lecturer: it offered hints, alternative perspectives and small pieces of information, while I determined the path of inquiry. In this setting, the quality of the learning process depended less on the model producing a complete answer and more on my ability to ask targeted questions and iteratively refine my understanding. As a consequence, even less capable models often proved sufficient, since their role was not to solve the problem for me but to sustain a dialogue in which I remained the primary agent of reasoning.

### Example: Brainstorming the Autograd Backward Pass

*The following is a condensed transcript of a real voice-command session. The casual, halting nature of the prompts has been preserved to show how raw spoken thoughts were refined into architectural abstractions.*

**Me (voice):** Alright, so for now I have this. If someone asked me “how do you implement the backward pass?”, I would say I need two things. I need a mechanism to compute the local step, and I need to know the actual *order* of the steps. But wait... couldn’t someone argue that the forward pass already gives me the order? Since we have a finite set of mechanisms performing these transformations, I don’t really need a new abstraction for the order, right?

**AI:** You are right that the order is given by the forward pass, but there is a conceptual trap. The chain rule dictates that we proceed in reverse:  $g_L \rightarrow g_{L-1} \rightarrow \dots \rightarrow g_1$ . Unlike the forward pass, the backward pass requires *memory*: you need the intermediate states ( $x$ ) computed during the forward pass in order to evaluate the local gradients. You need a way to record the history.

**Me (voice):** No, wait, slow down. I don’t care about the mathematical derivatives right now. I want to build my mental framework strictly through abstractions. Let’s stay concrete. I need exactly two things: to know *which* steps to perform, and to know *how* to perform each one. That’s it, right?

**AI:** You have hit the nail on the head. Architecturally, you need only two things:

1. **The local step abstraction:** how to take a gradient from the “future” and pass it to the “past”.
2. **The flow coordinator:** a mechanism that knows the topology and triggers those steps in reverse order.

**Me (voice):** Okay, keeping it concrete. To know *which* steps to perform: if the user has explicitly defined the order using our “composable transformations” abstraction, doesn’t that leave a trace? I’m getting lost. What even *is* that abstraction at runtime?

**AI:** A “composable transformation” is just an interface — a base type with a `forward()` method. By itself it computes the mathematics and immediately forgets the past. To leave a trace for the backward pass, it must actively record its execution onto a tape or a dynamic graph.

**Me (voice):** Stop. You need to be much, much shorter. This is a back-and-forth conversation; keep it brief, always. Okay, my question: is the “composable transformation” abstraction essentially just the implementation of a single layer? I called it a “finite set of mechanisms” only to keep it abstract.

**AI:** Yes, exactly. In code it is simply the `Layer` or `Module` interface. The “finite set” just means your basic building blocks: linear, ReLU, and so on.

**Me (voice):** Good. So, continuing my line of reasoning: if I build an architecture out of this finite set of layers, the architecture itself acts as a map. I can extract the backward order directly from the architecture’s own representation, right?

**AI:** Yes, with one distinction. If your graph is **static** (a fixed list of layers), the architecture *is* the map. If your graph is **dynamic** (if statements or loops inside the forward pass), the architecture is not enough: the map only comes into existence at runtime, by recording the trace.

**Me (voice):** Perfect. That distinction is exactly what I needed in order to design the coordinator...

This rapid, constrained back-and-forth was the methodology I applied across several complex parts of the program, and it suggests that AI is often most useful precisely when it is heavily restricted by the user.

#### 3.1.4 Cross-Checking Methodology: Validating the AI Output

A natural question arises: how did I cross-check the AI’s outputs to ensure correctness? The validation strategy depended entirely on whether I was using the AI for high-level reasoning, mathematical derivations, visual generation, or mechanical code translation.

For the reasoning and brainstorming phases I did *not* perform continuous, step-by-step cross-checking. Stopping to validate every micro-turn would have shattered the cognitive flow, particularly since I was the one steering the direction of the conversation. Instead, I built the logic iteratively, tracking my thoughts on paper. Once a complete line of reasoning was finished, and I felt reasonably confident in it, I performed a single, consolidated cross-check: I took the final conclusion to a fresh chat, or to an entirely different model, explicitly prompting it to look for availability bias inherited from the previous session.

Most of the time no hidden flaws emerged, because the framework was already structurally sound; Interrupting the process for mid-stream cross-checks would have been counterproductive.

When dealing with theoretical foundations tied directly to the curriculum, such as the discussion of polymorphism early in Chapter 2, my cross-checking relied strictly on the official course materials. To generate that section I provided the AI with a highly targeted prompt, in order to ensure academic rigour:

*“Act as an expert in programming language theory. Synthesise a formal explanation of polymorphism, explicitly distinguishing between ad hoc (overriding, overloading) and universal polymorphism. Contrast traditional class-based object-oriented approaches with trait-based paradigms (e.g. Rust). Conclude by framing universal polymorphism as the necessary theoretical foundation for achieving structural uniformity across neural network layers.”*

Once the AI had produced a draft, I manually cross-referenced its definitions and taxonomy against the course lecture notes and slides, ensuring strict alignment with the terminology taught in class.

For the mathematical derivations, specifically the backpropagation strategy and the forward/backward passes detailed in the appendix, a twofold validation was required. First, I passed the AI-generated derivations to a second model, tasked specifically with auditing subscript and indexing typos. Second, and more importantly, I performed a manual sanity check by explicitly tracking the dimensions of every matrix and vector at each step, guaranteeing that all dot products and element-wise operations were algebraically valid.

The generation of visual assets in TikZ required an iterative approach. The AI occasionally produced code that resulted in overlapping nodes or misaligned geometries. To validate and correct these, I compiled and inspected the renders manually. When a diagram lacked clarity, I sent the TikZ code to a different model together with the already-validated paragraph it was meant to illustrate, instructing it to reorganise the spatial layout so as to be as pedagogical as possible, with the visual topology mirroring the logic of the accompanying text.



### 3.2 Phase Two: From Specification to Implementation

The second phase begins where this document ends. Once the architecture was fixed, the nature of the work changed: it was no longer about deciding *what* to build, but about producing a large amount of code consistent with a specification I had already written. For this phase I deliberately changed tools.

#### 3.2.1 Setup: Claude Code as an Agentic Tool

For this part I subscribed to a Claude Pro plan and used **Claude Code** integrated with Visual Studio Code, so that the agent had direct access to the context of the repository rather than to isolated snippets pasted into a chat window. This was a significant departure from Phase One, where I had used only web interfaces.

The agent was configured through a `CLAUDE.md` file placed at the root of the project:

```
# Project Context

The file 'docs/architecture.pdf' contains the authoritative architectural specification
of this project.

Before making architectural decisions or implementing major components, consult this document.

The architecture described in the PDF is the fixed baseline of the project. Do not redesign
or contradict it unless explicitly instructed to do so.
```

The file `architecture.pdf` is essentially Chapter 2 of this document. Including more than one chapter would have been counterproductive: the remaining chapters do not bear directly on the implementation, and adding them would only have polluted the tool's context window with material irrelevant to the coding task.

#### 3.2.2 Validation Strategy

Instead of improvising a different check for each prompt, I applied a single, uniform validation procedure to all the code-generation prompts, which can be viewed in the 'RelevantPromptForCodeGeneration' PDF I uploaded. The procedure consists of three stages.

**1. High-level cross-check with a second model.** The output of every prompt was cross-checked with Gemini through the web interface, using the Pro model with extended reasoning. I attached `architecture.pdf` so that the reviewing model had access to the same authoritative specification given to the coding agent, and I supplied both the prompt and the resulting output in a message of the following form:

```
"Cross-check this work produced by Claude Code. Even though you can only evaluate it at a high level,
identify any significant issues, inconsistencies or potential weaknesses you can spot. If you do not find any,
state explicitly that you see no critical issues. Focus only on substantive concerns, not on minor stylistic
details.
PROMPT GIVEN TO CLAUDE: ...
OUTPUT PRODUCED: ..."
```

The instruction to state explicitly the absence of problems was deliberate: without it, review models tend to manufacture criticism in order to appear useful, which makes a clean result indistinguishable from a superficial one.

**2. Manual scan against the specification.** Regardless of the outcome of the automated review, I performed a rapid manual pass over the generated sources, checking that the module boundaries, the ownership model and the `Module` contract matched what Chapter 2 prescribes. This stage was concerned with architectural conformance, not with numerical correctness.

**3. Line-by-line review of the critical code.** For the parts carrying actual mathematical content, the backward passes, the losses and the optimizer updates, I reviewed the code line by line myself and relied on the test suite produced alongside it, verifying in particular shape consistency and the batch normalisation factor.

Two prompts produced findings worth reporting. For all the others, the cross-check returned no critical issues, and the manual scan did not reveal any divergence from the specification.

**Prompt 1 — project structure.** The review raised one observation concerning the `Sequential` container. I verified the point directly in the code and found the implementation to be correct, so no change was required; the observation is reported here for completeness, since a cross-check that only records defects gives a misleading picture of the process.

**Prompt 2 — layers and tensor operations.** Here the agent did not merely implement what I had asked: it also proposed a modification to the `Tensor` struct in order to support the stride-based operations required by the layers. Rather than accepting the suggestion implicitly, I examined it in depth to establish whether it was genuinely sound. It was, and I approved it, but since the specification is the baseline of the project, I also updated this document accordingly, so that the architecture and the implementation remained in agreement rather than silently diverging.

**Remaining prompts.** For the prompts covering losses and optimizers, the dataset handling and training loop, and the final experiments, neither the cross-check nor the manual review surfaced substantive concerns. In each case the reviewing model explicitly reported no critical issues.

### 3.2.3 Personal Reflections on the Second Phase

If in the first phase I insisted on total control, a phase that was, after all, about ideas, in the second I leaned forward and let go somewhat. The result left me unpleasantly surprised. I had never used an agentic tool before; I had always worked with AI through web interfaces, retaining full control. Here I watched the tool write code, run tests and generate an enormous quantity of code, so much that it became difficult even to follow.

I have to admit that all this capability saddened me a little. I originally chose computer science at university because programming fascinated me: I enjoyed spending hours on a problem, and I was captivated by the fact that the solution very often arrived precisely when I was not consciously thinking about it, in the shower, for instance. Much of that has been lost, and although I am still young, my enthusiasm had drifted elsewhere. I had hoped that this course, which I found genuinely curious and fascinating, would change my mind completely; but after trying this tool, I was left without words.

## 4 Appendix

### 4.1 Explicit derivation of the backward recursion

This appendix derives, in full, the recursion used in Section 1.2.2 to compute  $\nabla_{\theta} \ell(f_{\theta}(x), y)$  for  $f_{\theta} = g_L \circ \dots \circ g_1$ . Throughout,  $(x, y)$  is fixed,  $z_0 := x$ ,  $z_l := g_l(z_{l-1}; \theta_l)$ , and  $\ell(f_{\theta}(x), y) = \ell(z_L, y)$ .

**Chain rule (reminder).** If  $u$  depends on  $w$  only through an intermediate quantity  $v = v(w)$ , i.e.  $u = u(v(w))$ , then

$$\frac{\partial u}{\partial w} = \frac{\partial u}{\partial v} \cdot \frac{\partial v}{\partial w}. \quad (\text{CR})$$

When  $v$  and  $w$  are vectors,  $\partial u / \partial v$  and  $\partial v / \partial w$  are matrices of partial derivatives, and the product in (CR) is a matrix product, taken in the same order as the composition: the derivative with respect to the *outer* quantity ( $u$  w.r.t.  $v$ ) comes first, the derivative with respect to the *inner* quantity ( $v$  w.r.t.  $w$ ) comes second.

**Base case,  $L = 2$ .** Take

$$\underbrace{z_0 := x}_{\text{Input layer}}, \quad \underbrace{z_1 = g_1(z_0; \theta_1)}_{\text{Hidden layer}}, \quad \underbrace{z_2 = g_2(z_1; \theta_2)}_{\text{Output layer}}, \quad \underbrace{\theta = (\theta_1, \theta_2)}_{\text{Model parameters}}. \quad (9)$$

**Step 1 — forward pass.** Compute the numerical values needed, layer by layer, in the order dictated by the composition:

$$z_0 = x \xrightarrow{g_1(\cdot; \theta_1)} z_1 \xrightarrow{g_2(\cdot; \theta_2)} z_2 \xrightarrow{\ell(\cdot, y)} \ell(z_2, y). \quad (10)$$

Nothing is differentiated yet: this pass only produces the values  $z_1, z_2$  that the derivatives below will be evaluated at.

**Step 2 — backward pass.** We compute the pair  $(\nabla_{\theta_1} \ell(z_2, y), \nabla_{\theta_2} \ell(z_2, y))$ .

*Computing  $\nabla_{\theta_2} \ell(z_2, y)$ .* As a function of  $\theta_2$  alone,  $\ell(z_2, y)$  depends on  $\theta_2$  only through  $z_2$ :  $\ell$  is the outer quantity,  $z_2 = g_2(z_1; \theta_2)$  is the intermediate one. By (CR):

$$\frac{\partial \ell(z_2, y)}{\partial \theta_2} = \underbrace{\frac{\partial \ell(z_2, y)}{\partial z_2}}_{\text{outer, first}} \cdot \underbrace{\frac{\partial z_2}{\partial \theta_2}}_{\text{inner, second}}. \quad (11)$$

This is a row of partial derivatives; to match the usual (column) gradient notation  $\nabla_{\theta_2} \ell$ , transpose both sides — which reverses the order of the product:

$$\nabla_{\theta_2} \ell(z_2, y) = \left( \frac{\partial z_2}{\partial \theta_2} \right)^{\top} \nabla_{z_2} \ell(z_2, y) = \left( \frac{\partial g_2}{\partial \theta_2} \right)^{\top} \nabla_{z_2} \ell(z_2, y). \quad (12)$$

Since  $\nabla_{z_2} \ell(z_2, y)$  will reappear below, we name it:

$$a_2 := \nabla_{z_2} \ell(z_2, y), \quad \text{so that} \quad \nabla_{\theta_2} \ell(z_2, y) = \left( \frac{\partial g_2}{\partial \theta_2} \right)^{\top} a_2. \quad (\text{A.1})$$

*Computing  $\nabla_{\theta_1} \ell(z_2, y)$ .* Now  $\theta_1$  affects  $\ell$  through a longer chain,  $\theta_1 \rightarrow z_1 \rightarrow z_2 \rightarrow \ell$ , so (CR) is applied twice, again outer-to-inner:

$$\frac{\partial \ell(z_2, y)}{\partial \theta_1} = \underbrace{\frac{\partial \ell(z_2, y)}{\partial z_2}}_{\text{outermost}} \cdot \underbrace{\frac{\partial z_2}{\partial z_1}}_{\text{middle}} \cdot \underbrace{\frac{\partial z_1}{\partial \theta_1}}_{\text{innermost}}. \quad (13)$$

Transposing to recover the gradient reverses the order of the product:

$$\nabla_{\theta_1} \ell(z_2, y) = \left( \frac{\partial g_1}{\partial \theta_1} \right)^{\top} \underbrace{\left( \frac{\partial g_2}{\partial z_1} \right)^{\top} \nabla_{z_2} \ell(z_2, y)}_{= \nabla_{z_1} \ell(z_2, y)}. \quad (14)$$

The underbraced quantity is  $\nabla_{z_1} \ell(z_2, y)$ , obtained from  $a_2$  exactly as  $\nabla_{\theta_2} \ell$  was obtained from  $a_2$  above; we name it accordingly:

$$a_1 := \nabla_{z_1} \ell(z_2, y) = \left( \frac{\partial g_2}{\partial z_1} \right)^{\top} a_2, \quad \text{so that} \quad \nabla_{\theta_1} \ell(z_2, y) = \left( \frac{\partial g_1}{\partial \theta_1} \right)^{\top} a_1. \quad (\text{A.2--A.3})$$

The pattern is now visible: every gradient with respect to a parameter block is obtained by applying (CR) outer-to-inner and then transposing — and the transposition is exactly what turns "outer-to-inner" into the familiar "propagate an adjoint backward, one layer at a time" recursion, with  $a_l := \nabla_{z_l} \ell(z_2, y)$  collecting, at each stage, everything computed so far.

**Generalization to  $L$  layers, by induction.** We claim that, for a composition of any length  $L$ , with  $a_L := \nabla_{z_L} \ell(z_L, y)$ ,

$$a_{l-1} = \left( \frac{\partial g_l}{\partial z_{l-1}} \right)^\top a_l, \quad \nabla_{\theta_l} \ell(z_L, y) = \left( \frac{\partial g_l}{\partial \theta_l} \right)^\top a_l, \quad l = L, \dots, 1. \quad (\text{A.4})$$

*Base case,  $L = 2$ :* this is exactly (A.1)–(A.3) above.

*Inductive step.* Suppose the claim holds for a composition of  $L - 1$  layers,  $f = g_L \circ \dots \circ g_2$ : that is, starting from  $a_1 := \nabla_{z_1} \ell(z_L, y)$ , the recursion above correctly produces  $(\nabla_{\theta_2} \ell, \dots, \nabla_{\theta_L} \ell)$ . Now prepend one further layer  $g_1$ , so  $z_1 = g_1(z_0; \theta_1)$ . As a function of  $\theta_1$  alone,  $\ell(z_L, y)$  depends on  $\theta_1$  only through  $z_1$ , so (CR) gives, exactly as in the  $\theta_2$  computation above,

$$\nabla_{\theta_1} \ell(z_L, y) = \left( \frac{\partial g_1}{\partial \theta_1} \right)^\top \nabla_{z_1} \ell(z_L, y) = \left( \frac{\partial g_1}{\partial \theta_1} \right)^\top a_1, \quad (15)$$

where  $\nabla_{z_1} \ell(z_L, y) = a_1$  is exactly the seed the inductive hypothesis already assumed. Layers  $2, \dots, L$  are untouched, so their gradients follow from the inductive hypothesis unchanged. This establishes the claim for  $L$  layers, completing the induction.

**Resulting procedure.**

|                                                                   |                                                                                             |
|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>seed:</b>                                                      | $a_L := \nabla_{z_L} \ell(z_L, y)$                                                          |
| <b>for <math>l = L, L - 1, \dots, 1</math>, repeat two steps:</b> |                                                                                             |
| <i>read off this layer's gradient:</i>                            | $\nabla_{\theta_l} \ell(z_L, y) = \left( \partial g_l / \partial \theta_l \right)^\top a_l$ |
| <i>propagate to the previous layer:</i>                           | $a_{l-1} = \left( \partial g_l / \partial z_{l-1} \right)^\top a_l$                         |

This is exactly (A.4): nothing new has been postulated, it is (CR) applied outer-to-inner and then transposed, once per layer,  $L$  times in total.

## 4.2 Algebraic Complexity and Dimensionality of VJP vs. JVP Accumulation

### Forward Accumulation

(Left-to-Right / Jacobian-Vector Products)

Evaluates (7') by grouping the matrix products starting from the parameters (left) and sweeping **forward** toward the loss (right):

$$\underbrace{\left(\frac{\partial g_{l+1}}{\partial z_l}\right) \dots \left(\frac{\partial g_L}{\partial z_{L-1}}\right)}_{d_L \times d_l} \underbrace{\left[\left(\frac{\partial g_l}{\partial \theta_l}\right)v\right]}_{d_l \times 1}$$

**Why not propagate the full Jacobian?** Define  $J_j := \partial z_j / \partial \theta_l \in \mathbb{R}^{d_j \times p_l}$  and unroll the recursion:

$$\begin{aligned} J_l &= \frac{\partial g_l}{\partial \theta_l} \in \mathbb{R}^{d_l \times p_l} \quad (\text{seed}) \\ J_{l+1} &= \underbrace{\frac{\partial g_{l+1}}{\partial z_l}}_{d_{l+1} \times d_l} \underbrace{J_l}_{d_l \times p_l} \in \mathbb{R}^{d_{l+1} \times p_l} \\ &\vdots \\ J_L &= \underbrace{\frac{\partial g_L}{\partial z_{L-1}}}_{d_L \times d_{L-1}} \underbrace{J_{L-1}}_{d_{L-1} \times p_l} \in \mathbb{R}^{d_L \times p_l} \end{aligned}$$

One sweep already gives  $J_L = \partial z_L / \partial \theta_l$  in full — no repetition needed. But each step multiplies a  $d_j \times d_{j-1}$  matrix by a  $d_{j-1} \times p_l$  **matrix**, not a vector:

$$\underbrace{\left(\frac{\partial g_j}{\partial z_{j-1}}\right)}_{d_j \times d_{j-1}} \underbrace{\dot{z}_{j-1}}_{d_{j-1} \times 1} = \underbrace{\dot{z}_j}_{d_j \times 1} \quad (\text{single direction})$$

$$\underbrace{\left(\frac{\partial g_j}{\partial z_{j-1}}\right)}_{d_j \times d_{j-1}} \underbrace{J_{j-1}}_{d_{j-1} \times p_l} = \underbrace{J_j}_{d_j \times p_l} \quad (p_l \times \text{ heavier})$$

Cost per step:  $\mathcal{O}(d_j d_{j-1})$  vs.  $\mathcal{O}(d_j d_{j-1} p_l)$  — and  $J_{j-1}$  must be held in memory in full. Forward mode avoids this by collapsing each  $J_j$  to a single column,  $v = e_k$ , propagating only  $\dot{z}_j := J_j e_k \in \mathbb{R}^{d_j}$ :

$$J_j \in \mathbb{R}^{d_j \times p_l} \xrightarrow{\text{fix column } k} \dot{z}_j = J_j e_k \in \mathbb{R}^{d_j}.$$

Each step is now cheap (matrix-vector), but the sweep knows only column  $k$  of  $J_L$ : recovering the full gradient means repeating it for  $k = 1, \dots, p_l$ .

**Step-by-step sweep, fixed direction  $v = e_k$  at layer  $l$ .**  $L - l + 1$  steps, each a single JVP:

- **Seed:**  $\dot{z}_l = \underbrace{\left(\frac{\partial g_l}{\partial \theta_l}\right)}_{d_l \times p_l} \underbrace{e_k}_{p_l \times 1}$ .
- **Step  $j = l + 1, \dots, L$ :**  $\dot{z}_j = \underbrace{\left(\frac{\partial g_j}{\partial z_{j-1}}\right)}_{d_j \times d_{j-1}} \underbrace{\dot{z}_{j-1}}_{d_{j-1} \times 1}$ .
- **Termination:**  $\dot{z}_L \in \mathbb{R}^{d_L}$  is  $\partial \ell / \partial \theta_{l,k}$  (up to contraction with  $\nabla_{z_L} \ell$ ) — the directional derivative along component  $k$  of  $\theta_l$ .

**Sweeps:  $p_l$ , this layer only.**  $p_l$  sweeps (one per  $e_k$ ) seeded at  $l$  give  $\nabla_{\theta_l} \ell$ .

**Other layers? No.**  $p = \sum_l p_l$ : fresh re-seed per layer, sharing nothing. Unequal cost too — sweep at  $l$  walks  $L - l + 1$  steps. total cost =  $\sum_{l=1}^L p_l (L - l + 1) \cdot \mathcal{O}(d \cdot d)$ . Reverse: one fixed-length- $L$  sweep gets *all* layers at once.

### Reverse Accumulation (Backprop)

(Right-to-Left / Vector-Jacobian Products)

**Already formalized above; repeated here for completeness and side-by-side comparison with forward mode.**

Evaluates (7') by grouping the matrix products starting from the scalar loss (right) and sweeping **backward** toward the parameters (left):

$$\underbrace{\left(\frac{\partial g_l}{\partial \theta_l}\right)^\top}_{p_l \times d_l} \underbrace{\left[\left(\frac{\partial g_{l+1}}{\partial z_l}\right)^\top \dots \left(\frac{\partial g_L}{\partial z_{L-1}}\right)^\top a_L\right]}_{d_l \times 1}$$

**Why this problem never arises here.** No analogue of the forward dilemma — matrix-matrix (expensive) vs. matrix-vector repeated  $p$  times — exists in reverse mode, by the shape of the problem itself:

$$a_l := \frac{\partial \ell}{\partial z_l} \in \mathbb{R}^{d_l \times 1}, \quad \ell \in \mathbb{R}^1$$

Since  $\dim(\ell) = 1$ , the object being propagated is *already* a single vector — never one column of a wider  $d_L \times d_l$  Jacobian that forward mode must fix and repeat over:

$$\underbrace{\frac{\partial z_L}{\partial \theta_l}}_{\text{forward: } d_L \times p_l, \text{ must widen}} \quad \text{vs.} \quad \underbrace{\frac{\partial \ell}{\partial z_l}}_{\text{reverse: } d_l \times 1, \text{ already a vector}}$$

So every step is already matrix-vector, with nothing to fix at a time and nothing to reassemble column by column:

$$\underbrace{\left(\frac{\partial g_{j+1}}{\partial z_j}\right)^\top}_{d_j \times d_{j+1}} \underbrace{a_{j+1}}_{d_{j+1} \times 1} = \underbrace{a_j}_{d_j \times 1} \quad (\text{one sweep, all } p_l \text{ components at once})$$

**Step-by-step sweep.** One reverse sweep,  $L$  steps, each a single VJP:

- **Seed ( $l = L$ ):**

$$a_L := \nabla_{z_L} \ell(z_L, y) \in \mathbb{R}^{d_L}.$$

- **Step  $l = L, L - 1, \dots, 1$ , given  $a_l \in \mathbb{R}^{d_l}$ :**

$$\underbrace{\nabla_{\theta_l} \ell}_{p_l \times 1} = \underbrace{\left(\frac{\partial g_l}{\partial \theta_l}\right)^\top}_{p_l \times d_l} \underbrace{a_l}_{d_l \times 1} \quad (\text{local gradient})$$

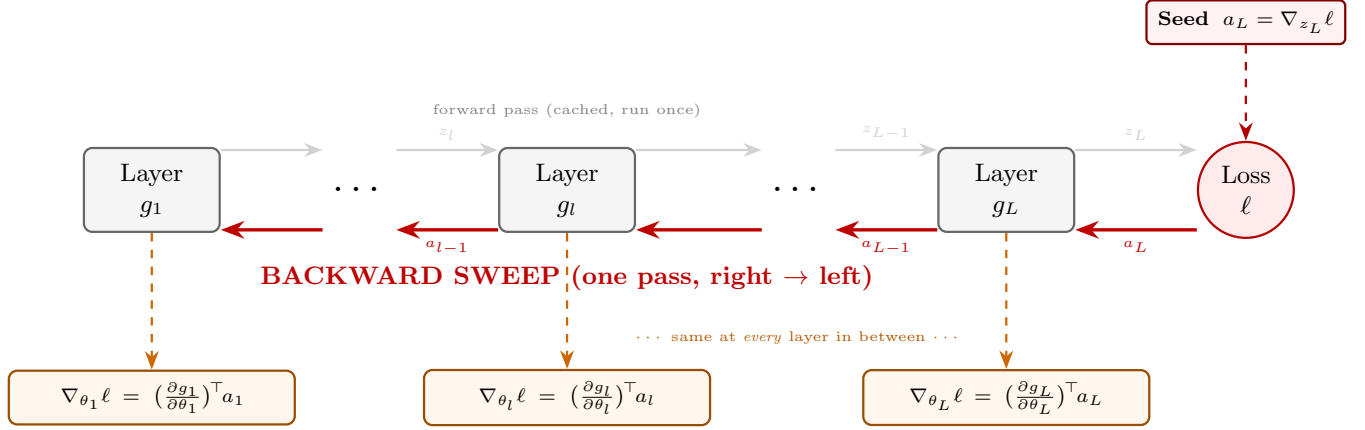
$$\underbrace{a_{l-1}}_{d_{l-1} \times 1} = \underbrace{\left(\frac{\partial g_l}{\partial z_{l-1}}\right)^\top}_{d_{l-1} \times d_l} \underbrace{a_l}_{d_l \times 1} \quad (\text{adjoint})$$

- **Termination:** after step  $l = 1$ , all  $L$  local gradients  $\{\nabla_{\theta_l} \ell\}_{l=1}^L$  have been produced — *every layer, in this one pass* — and their concatenation is  $\nabla_{\theta} \ell \in \mathbb{R}^p$ .

**Number of sweeps: 1.** At every step, only a  $d_{l-1} \times d_l$  matrix meets a  $d_l \times 1$  vector: never a matrix-matrix product, and never a re-derivation of an earlier step. A single reverse sweep produces the gradient with respect to *all*  $p$  parameters, across *all*  $L$  layers, at once — no layer is left for a separate pass.

# Reverse Mode

Backward accumulation (VJP) — one sweep, every layer

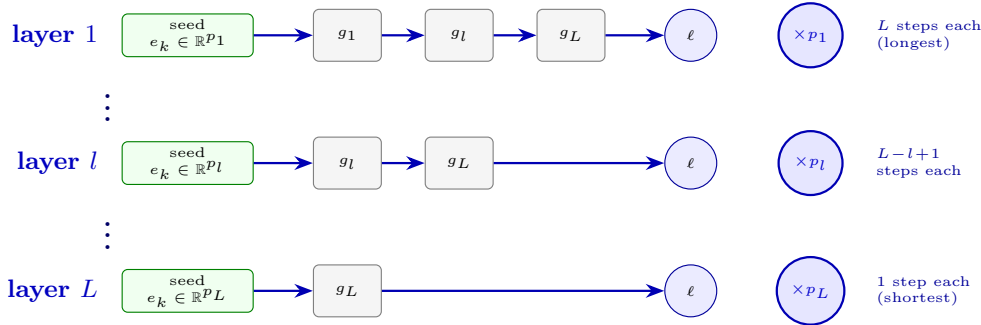


**ONE BACKWARD SWEEP  $\Rightarrow$  ALL  $L$  LOCAL GRADIENTS, PRODUCED TOGETHER**

$\mathcal{O}(1)$  sweep, fixed length  $L$ . The adjoint chain  $a_L \rightarrow a_{L-1} \rightarrow \dots \rightarrow a_1$  visits every layer exactly once; at each visited layer the local gradient is peeled off “for free” along the way — no layer needs a dedicated pass of its own.

# Forward Mode

Forward accumulation (JVP) — a separate set of sweeps per layer



**EVERY LAYER NEEDS ITS OWN, INDEPENDENT SET OF SWEEPS**

**Algorithm.** for  $l = 1, \dots, L$ : for  $k = 1, \dots, p_l$ : run one forward sweep seeded at  $(l, k)$ , of length  $L - l + 1$ . Total  $p = \sum_l p_l$  sweeps, of *unequal* length — independent across both  $k$  and  $l$ , hence parallelizable, but never as cheap in aggregate as the single reverse sweep opposite.

## Legend

$\longrightarrow$  forward pass (cached)  $\longrightarrow$  VJP (reverse)  $\longrightarrow$  JVP (forward)  $\dashrightarrow$  gradient extracted  $\dashrightarrow$  seed / fixed direction

## Cost Summary

|               | sweeps           | cost / step                | total cost                                        |
|---------------|------------------|----------------------------|---------------------------------------------------|
| Reverse (VJP) | 1                | $\mathcal{O}(d_j d_{j-1})$ | $\mathcal{O}(d_j d_{j-1})$                        |
| Forward (JVP) | $p = \sum_l p_l$ | $\mathcal{O}(d_j d_{j-1})$ | $\mathcal{O}(\sum_l p_l (L - l + 1) d_j d_{j-1})$ |

Figure 6: Comparison of reverse- and forward-mode automatic differentiation. **Left (Reverse Mode):** a single backward sweep, seeded once at the loss, visits every layer exactly once; the local gradients  $\nabla_{\theta_1} \ell, \nabla_{\theta_l} \ell, \nabla_{\theta_L} \ell$  (and every other layer’s) are all peeled off *within that same pass* ( $\mathcal{O}(1)$  sweep, fixed length  $L$ ). **Right (Forward Mode):** each layer must be seeded and swept *independently* —  $p_l$  sweeps for layer  $l$  alone, of length  $L - l + 1$  — so recovering the full gradient requires  $p = \sum_l p_l$  separate sweeps of unequal cost, as made explicit by the **for-for** algorithm and confirmed by the summary table below.